

Regexes and Denial of Service

Sophie Chen

HOW TO REGEX

STEP 1: OPEN YOUR FAVORITE EDITOR



STEP 2: LET YOUR CAT PLAY ON YOUR KEYBOARD



Agenda

1.

What is a regex?

2.

How are regexes
implemented?

3.

How can regexes
lead to DoS?

Agenda

1.

~~What is a regex?~~

Computability
theory

2.

~~How are regexes
implemented?~~

Feature creep

3.

~~How can regexes
lead to DoS?~~

Cloudflare outage

Agenda

1.

What is a regex?

2.

How are regexes
implemented?

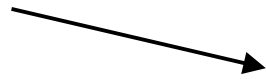
3.

How can regexes
lead to DoS?

What is a regex?

Informally, a regular expression is a sequence of characters that matches a pattern in text.

matches "hello"



hello
[a-zA-Z]*
(c|b)+at

matches "asdf",
"", "E"

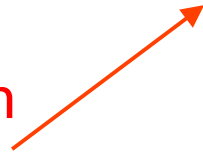


matches "cat",
"bat", "cbat"



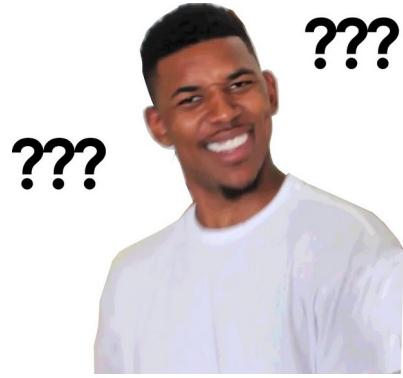
What problems are solvable by computers?

We will not answer this in
this presentation :)



What is a problem?

We consider decision problems, where the goal is to decide (yes or no) if a given input **object** has a certain **property**.



What problems are solvable by computers?

Alphabet (denoted Σ): A set of symbols

$$\Sigma_{\text{abs}} = \{a, b\}$$

String: A sequence of symbols from an alphabet ^[1]

aabb

Language (denoted L): A set of strings

{abab, a, abbbbbbb}

What problems are solvable by computers?

Problem: We consider decision problems, where the goal is to decide (yes or no) if a given input **object** has a certain property.

Any finite object can be encoded as a finite string x of characters from an alphabet Σ ^[2].

- Length of a string x is denoted $|x|$
- Empty string is denoted ε

What problems are solvable by computers?

Problem: We consider decision problems, where the goal is to decide (yes or no) if a given input object has a certain **property**.

A property corresponds to a language where the language contains all the "yes"es of the property ^[2].

$$L_{as} = \{a, aa, aaa, aaaa, \dots\}$$

What problems are solvable by computers?

What is a problem?

We consider decision problems, where the goal is to decide (yes or no) if a given input **object** has a certain **property**.

What problems are solvable by computers?

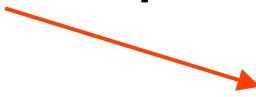
What is a computer?

Models of computation include finite automata, Turing machines, lambda calculus, etc.

What problems are solvable by computers?

What is a ~~computer~~?

Models of computation include **finite automata**, Turing machines, lambda calculus, etc.



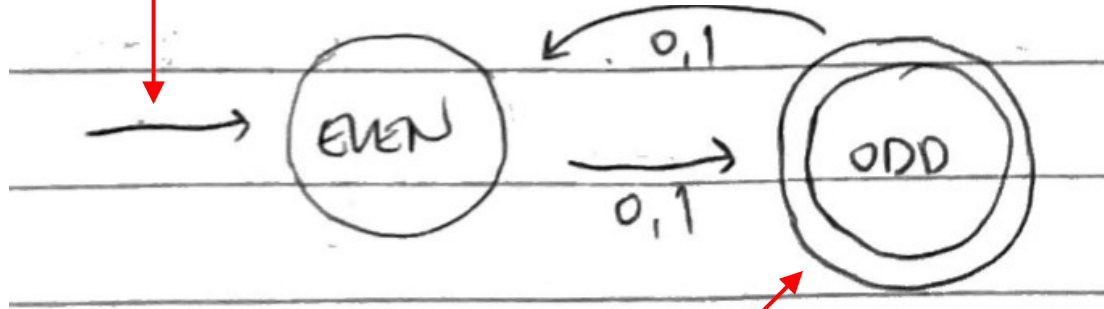
What are deterministic finite automata?

A DFA consists of the following five elements ^[2]:

- (1) Q : A finite set of states
- (2) q_0 : The initial state
- (3) Σ : The input alphabet
- (4) δ : A transition function
- (5) F : A finite set of “yes” states (a subset of Q)

What are deterministic finite automata?

indicates
initial state

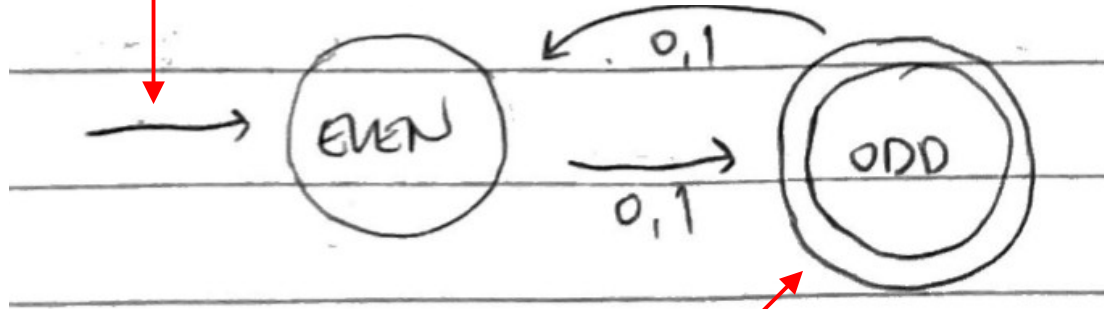


indicates "yes"

- (1) Q : A finite set of states
- (2) q_0 : The initial state
- (3) Σ : The input alphabet
- (4) δ : A transition function
- (5) F : A finite set of "yes" states

What are deterministic finite automata?

indicates
initial state

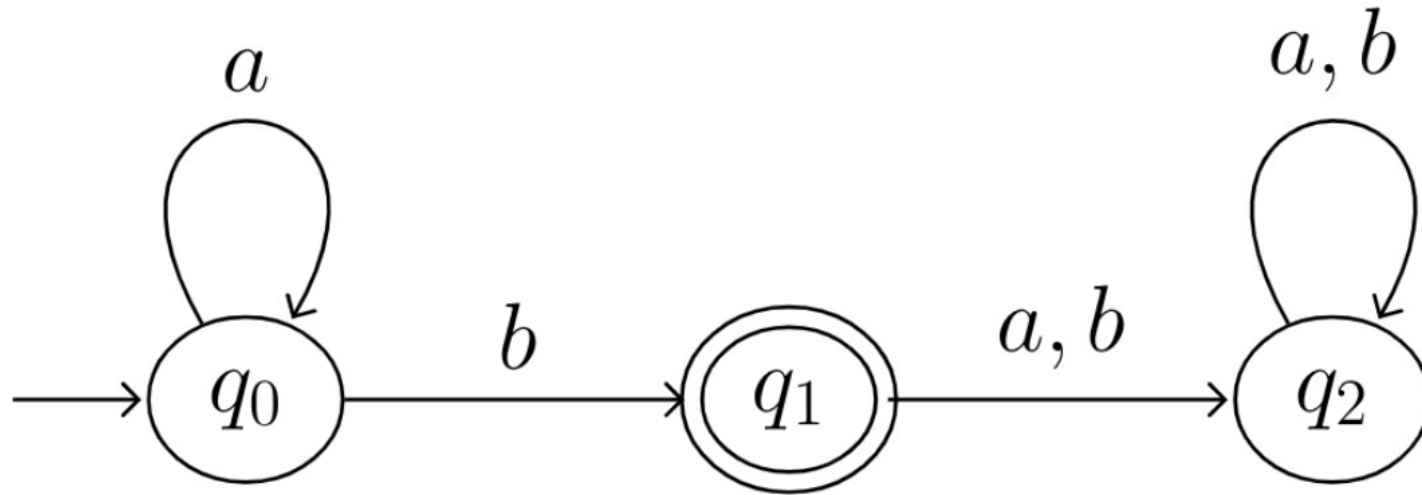


indicates "yes"

A DFA that decides whether $|x|$ is odd for a given string x , where $\Sigma = \{0, 1\}$

What are deterministic finite automata?

Practice: Think about a DFA that decides whether x is of the form a^*b , where $\Sigma = \{a, b\}$ ^[1]



What are nondeterministic finite automata?

NFA are very similar to DFA, except they can:

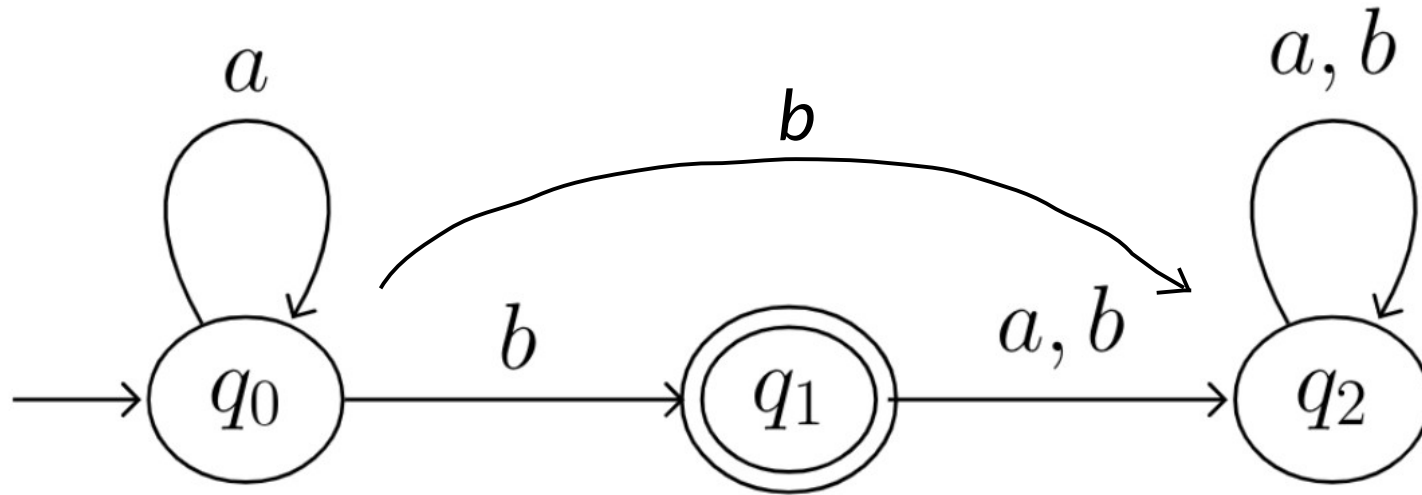
1. Have multiple possible transitions between states for the same input symbol ^[8]
2. Transition between states without consuming an input symbol ^[8]

NFAs have been proven to have the same expressivity as DFAs.



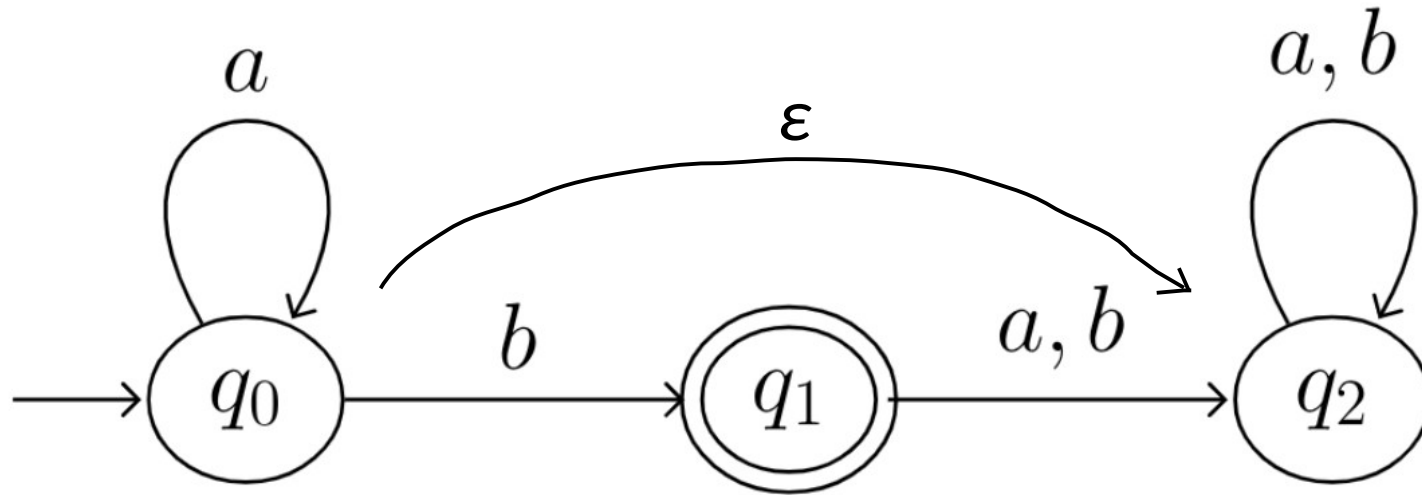
What are nondeterministic finite automata?

Multiple possible transitions between states for the same input symbol ^[8]



What are nondeterministic finite automata?

Transition between states without consuming an input symbol^[8]



What is a regex?

Formally, a Kleene regex (defined as a set) is made up of ^[4]:

- $\{\}$
- ϵ
- A literal character in Σ

Operator	Description
ab	Concatenation
$a b$	Disjunction
a^*	Kleene star

What is a regex?

Let our alphabet $\Sigma = \{a, b\}$.

$(a|b)b^*$ is a regex that matches _____.

Operator	Description
ab	Concatenation
$a b$	Disjunction
a^*	Kleene star

What is a regex?

Let our alphabet $\Sigma = \{a, b\}$.

$(a|b)b^*$ is a regex that matches a , b , one or more b 's or an a followed by one or more b 's.

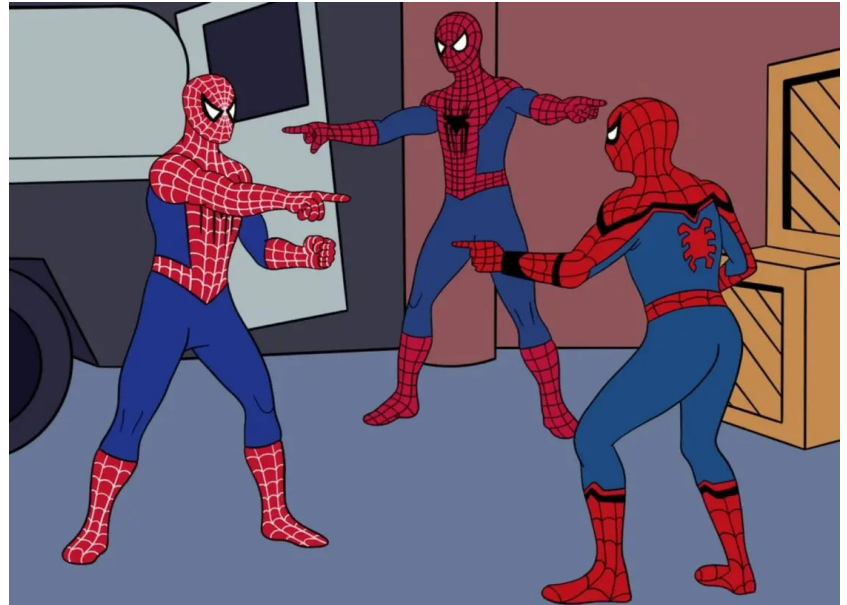
Operator	Description
ab	Concatenation
$a b$	Disjunction
a^*	Kleene star

What is a regex?

A regular language is a language that can be decided by a DFA ^[1].

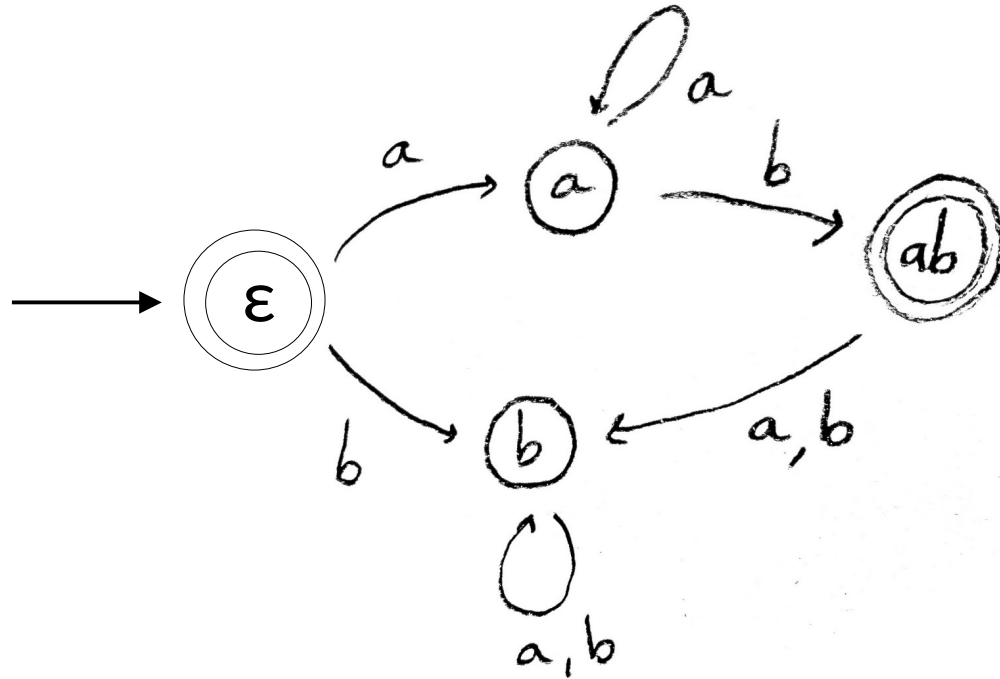
← and by extension, an NFA

Regexes express regular languages while DFAs determine regular expressions.



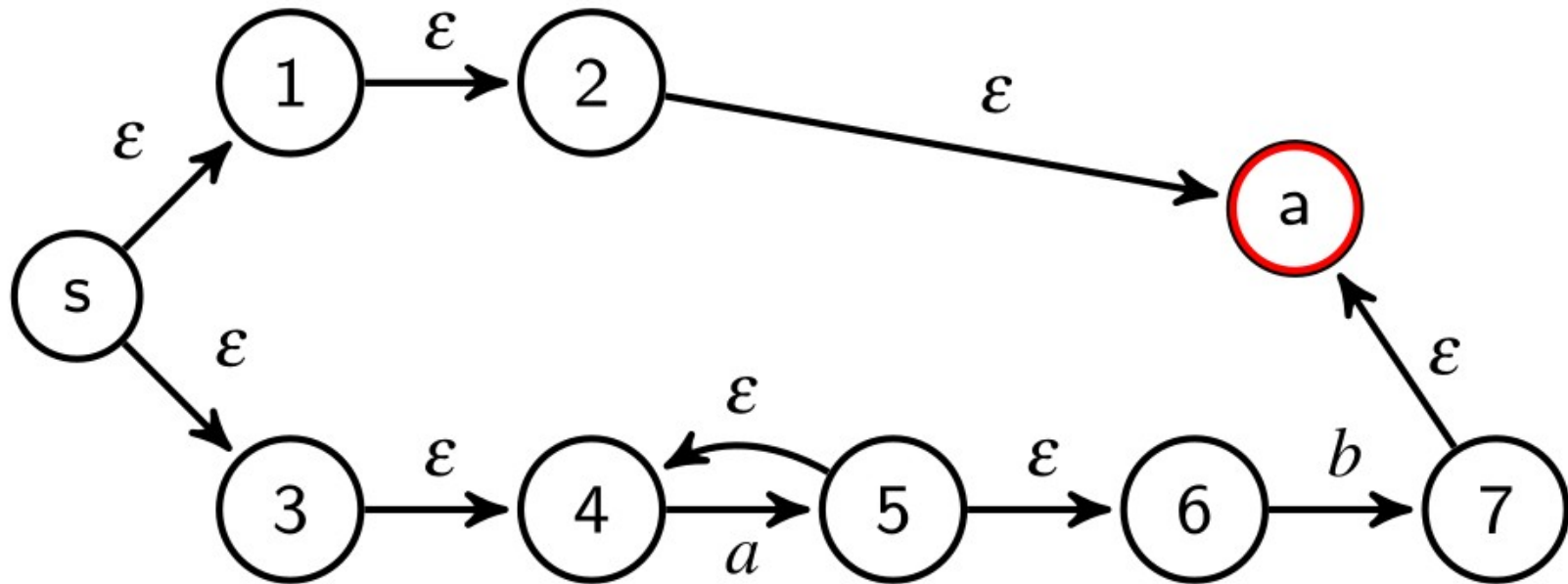
What is a regex?

Consider $\Sigma = \{a, b\}$ and the regex $(aa^*b) | \epsilon$



What is a regex?

Consider $\Sigma = \{a, b\}$ and the regex $(aa^*b) | \epsilon$



Agenda

1.

What is a regex?

2.

**How are regexes
implemented?**

3.

How can regexes
lead to DoS?

How are regexes implemented?

Multiple approaches for for evaluating regexes:

1. Convert into a DFA
2. Convert into an NFA and use backtracking (Spencer) [5, 7]
3. Convert into an NFA and use concurrent search (Thompson) [5, 6]
4. Some combination of 1-3

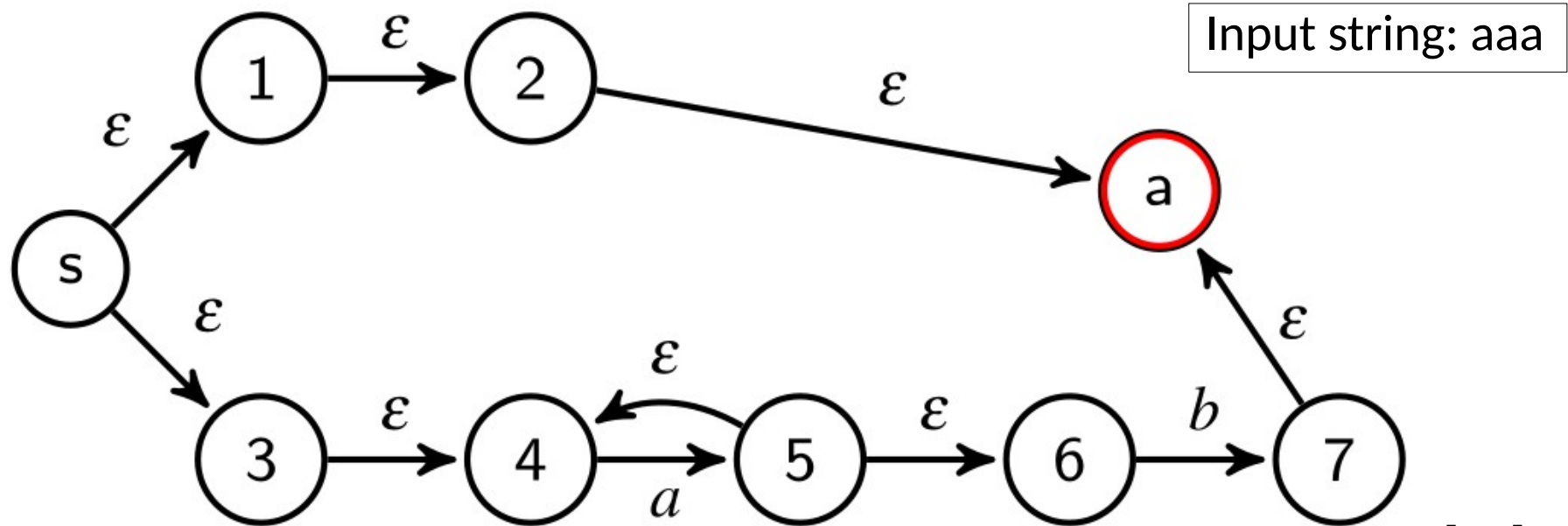
How are regexes implemented?

Multiple approaches for for evaluating regexes:

1. Convert into a DFA
- 2. Convert into an NFA and use backtracking (Spencer) [5, 7]**
- 3. Convert into an NFA and use concurrent search (Thompson) [5, 6]**
4. Some combination of 1-3

How does Spencer's algorithm work?

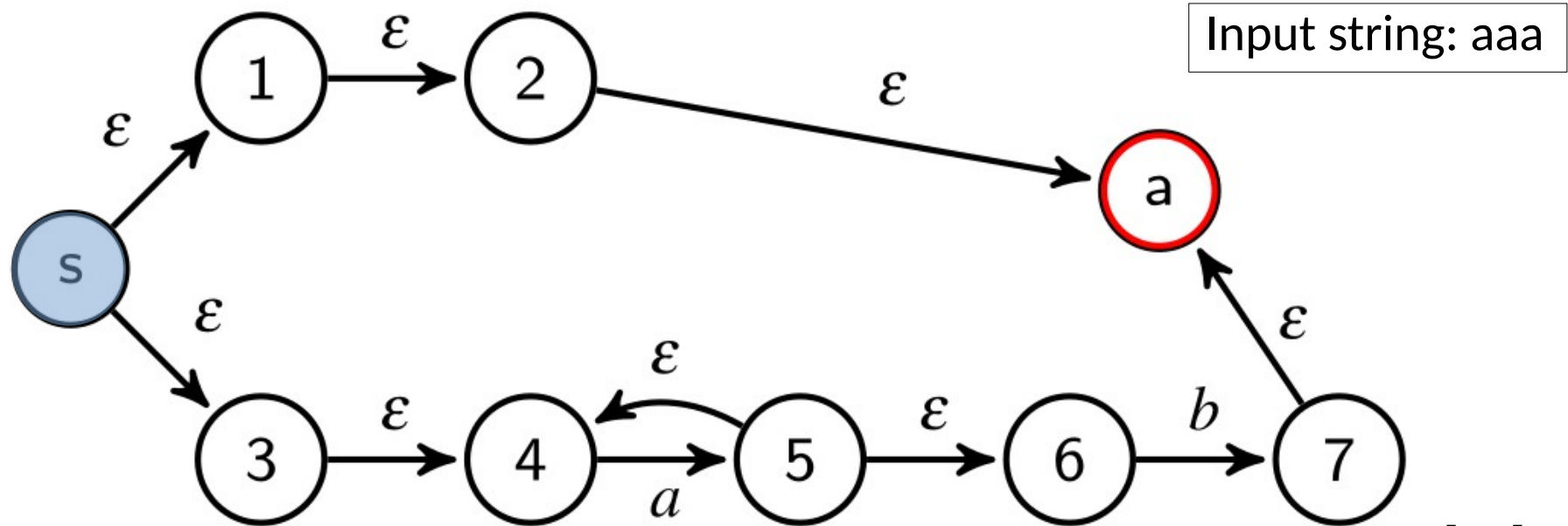
Convert $(aa^*b) | \epsilon$ into an NFA and then use backtracking



Input string: aaa

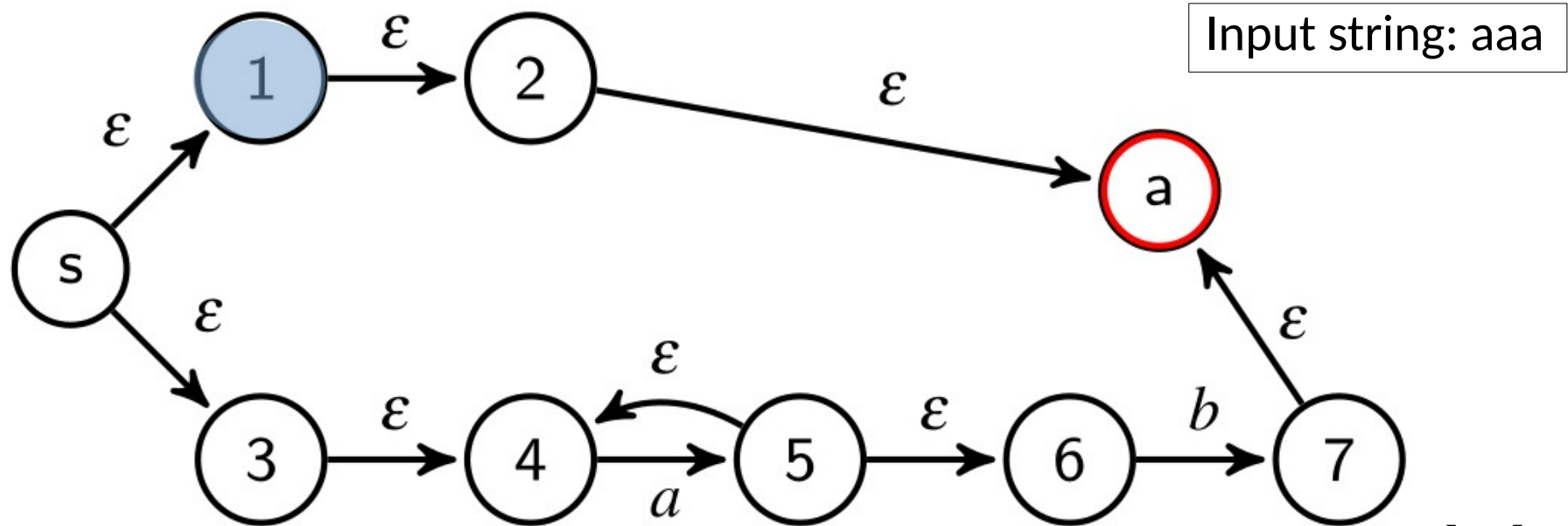
How does Spencer's algorithm work?

Convert $(aa^*b) \mid \epsilon$ into an NFA and then use backtracking



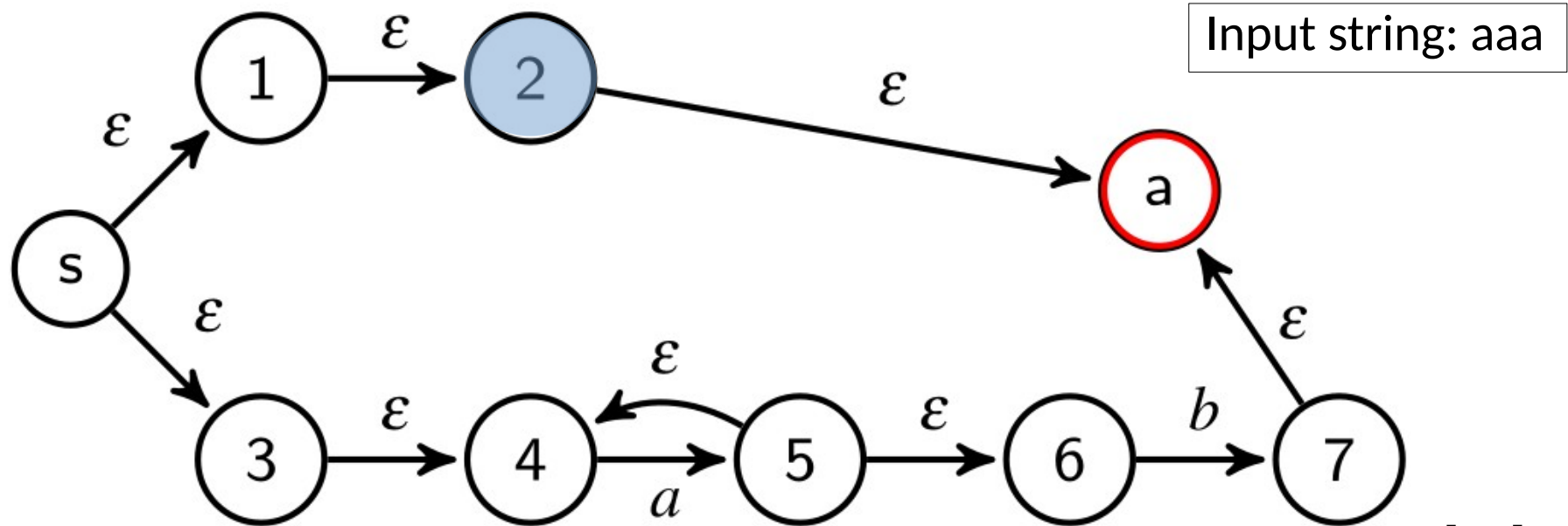
How does Spencer's algorithm work?

Convert $(aa^*b) \mid \epsilon$ into an NFA and then use backtracking



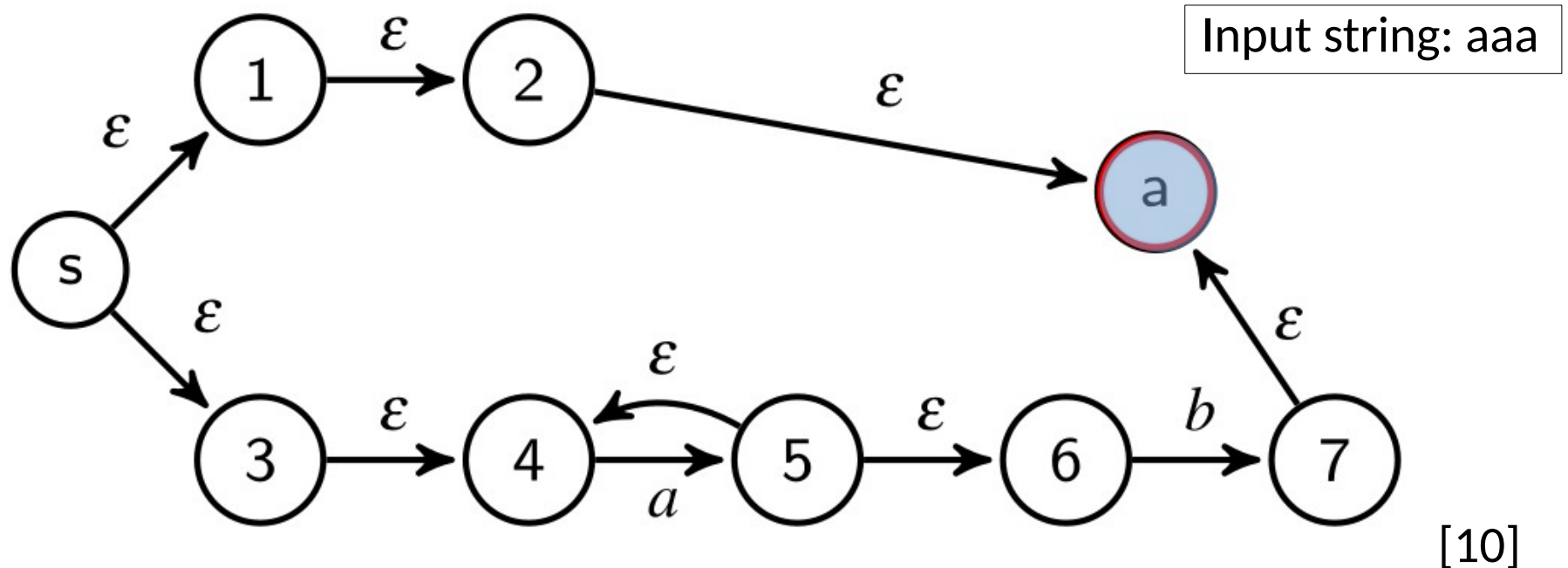
How does Spencer's algorithm work?

Convert $(aa^*b) \mid \epsilon$ into an NFA and then use backtracking



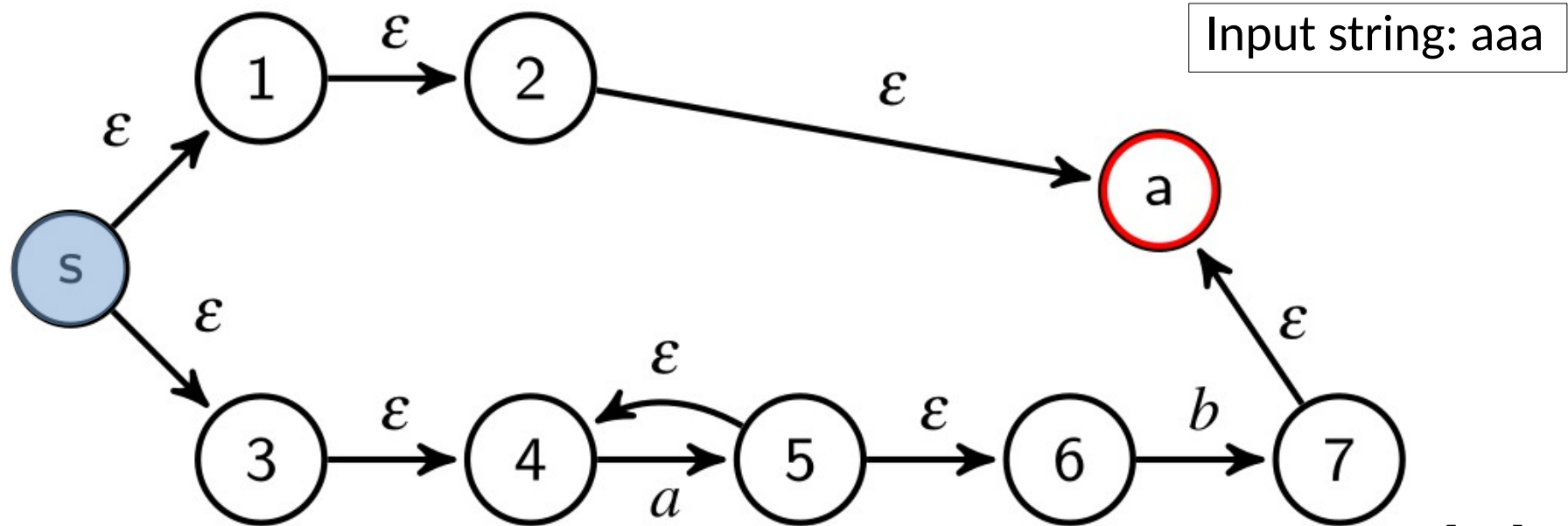
How does Spencer's algorithm work?

Convert $(aa^*b) \mid \epsilon$ into an NFA and then use backtracking



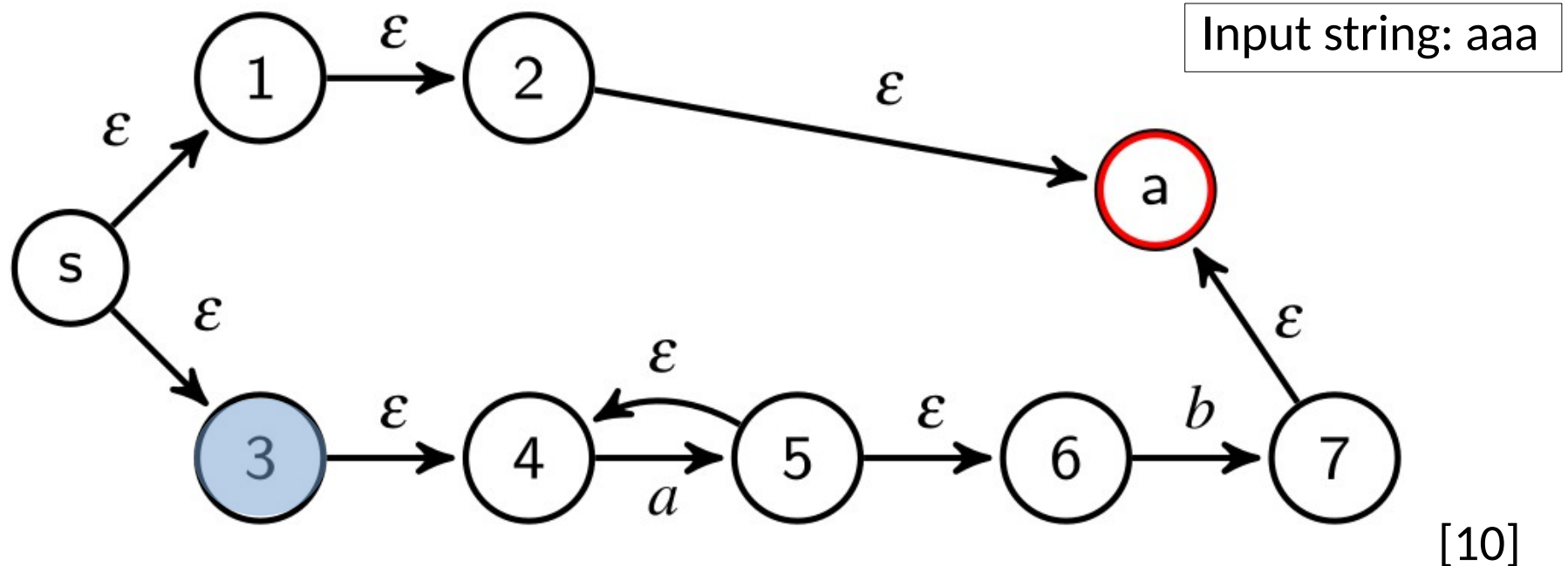
How does Spencer's algorithm work?

Convert $(aa^*b) \mid \epsilon$ into an NFA and then use backtracking



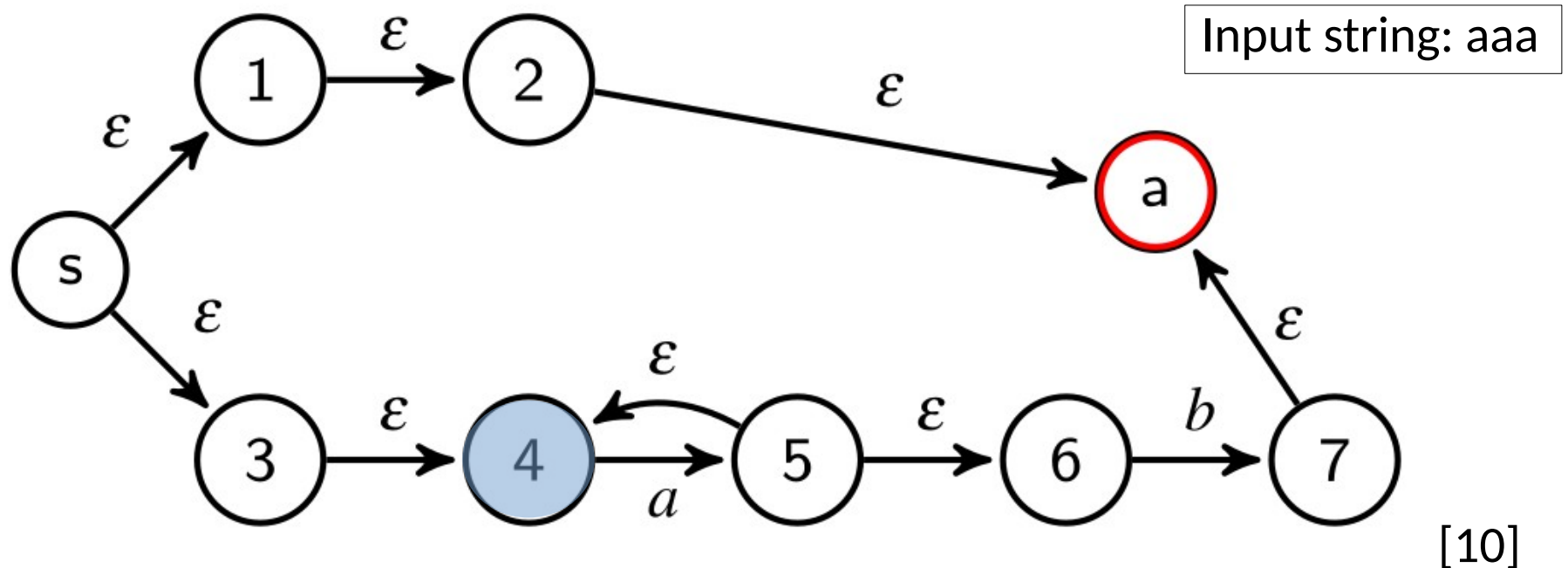
How does Spencer's algorithm work?

Convert $(aa^*b) \mid \epsilon$ into an NFA and then use backtracking



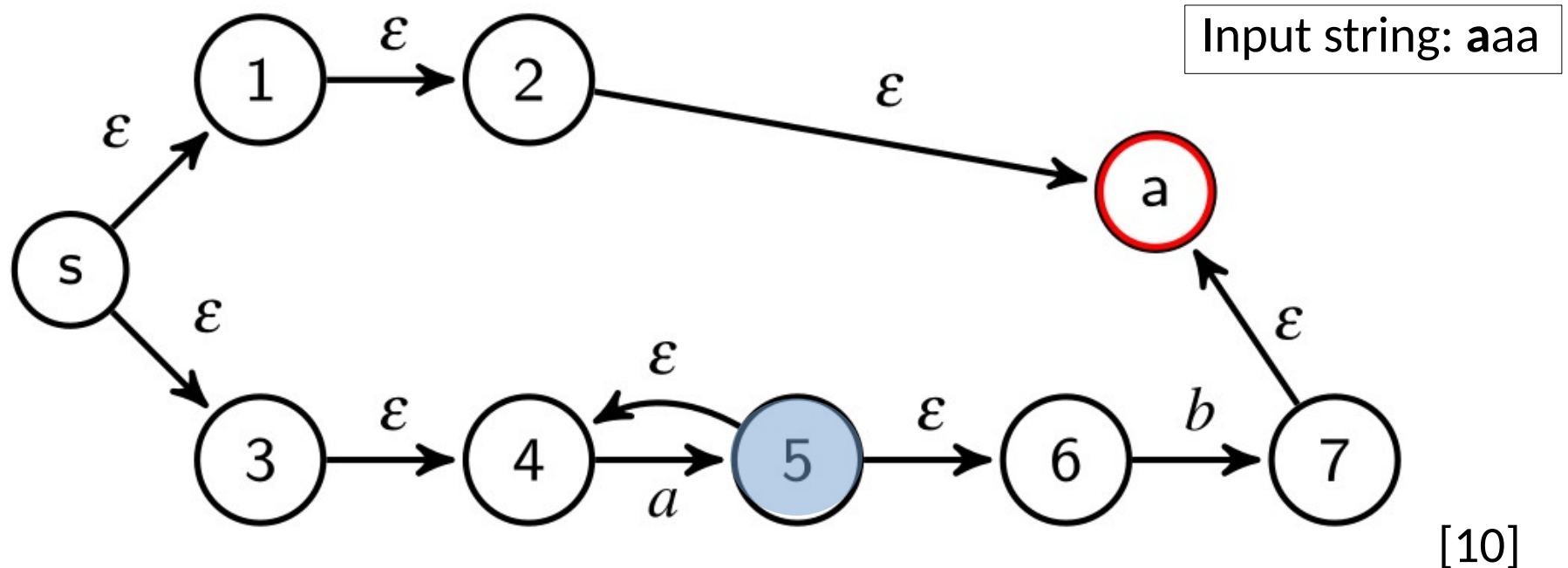
How does Spencer's algorithm work?

Convert $(aa^*b) | \epsilon$ into an NFA and then use backtracking



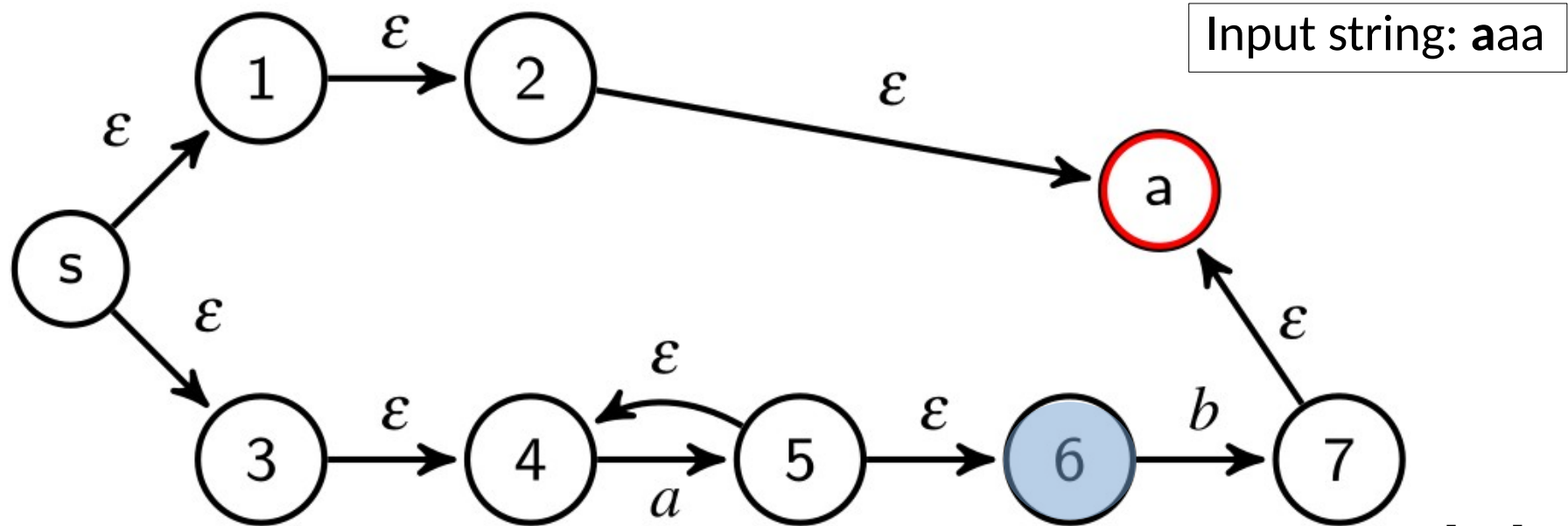
How does Spencer's algorithm work?

Convert $(aa^*b) | \epsilon$ into an NFA and then use backtracking



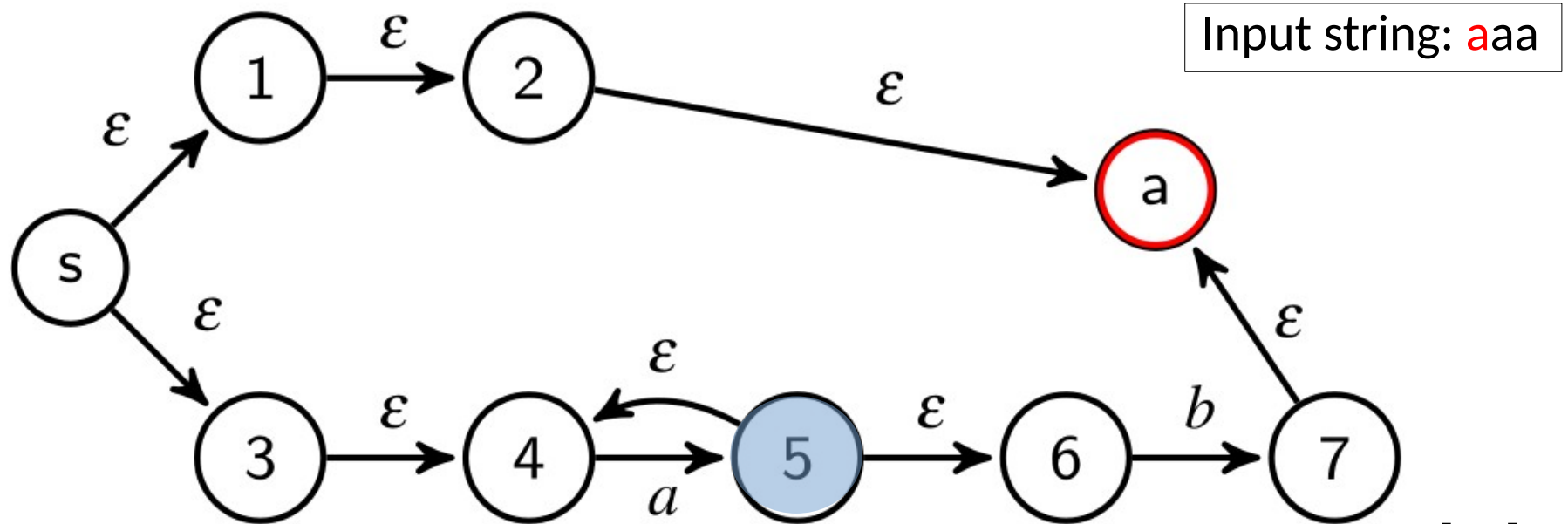
How does Spencer's algorithm work?

Convert $(aa^*b) \mid \epsilon$ into an NFA and then use backtracking



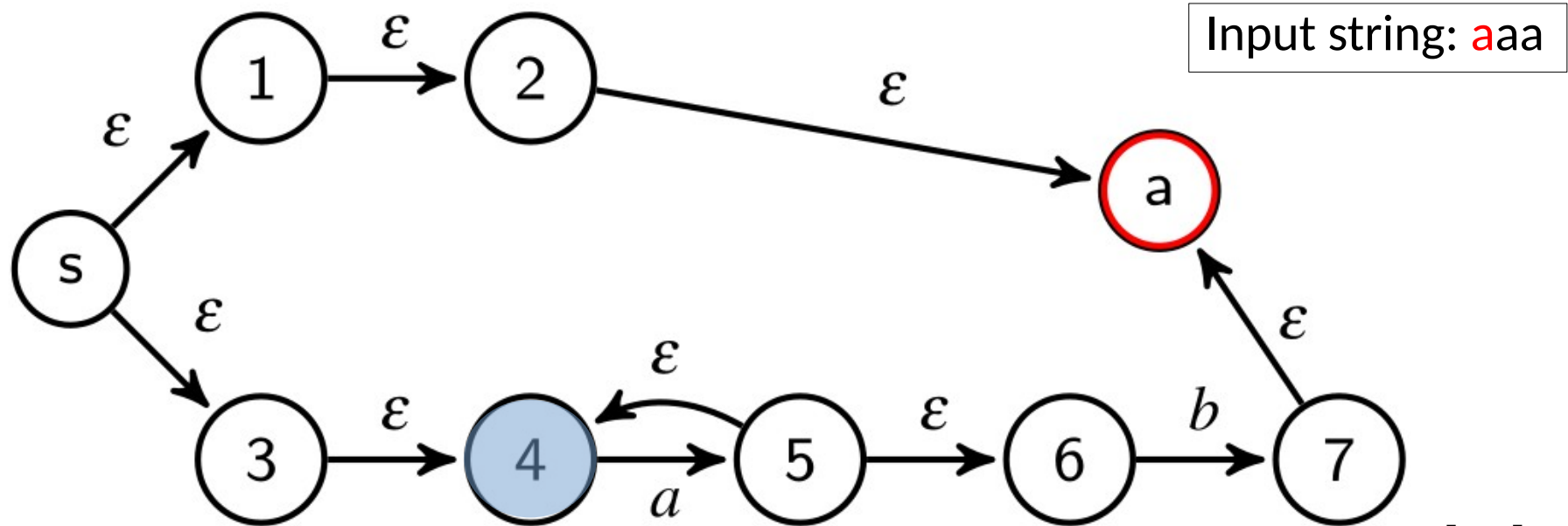
How does Spencer's algorithm work?

Convert $(aa^*b) | \epsilon$ into an NFA and then use backtracking



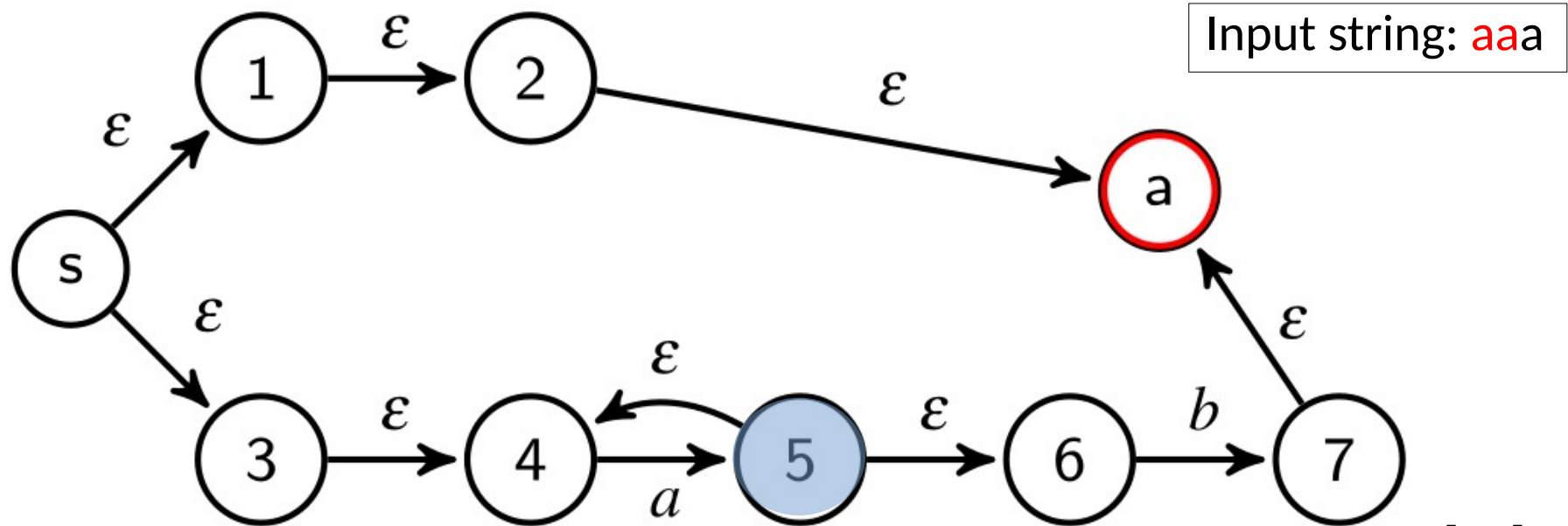
How does Spencer's algorithm work?

Convert $(aa^*b) \mid \epsilon$ into an NFA and then use backtracking



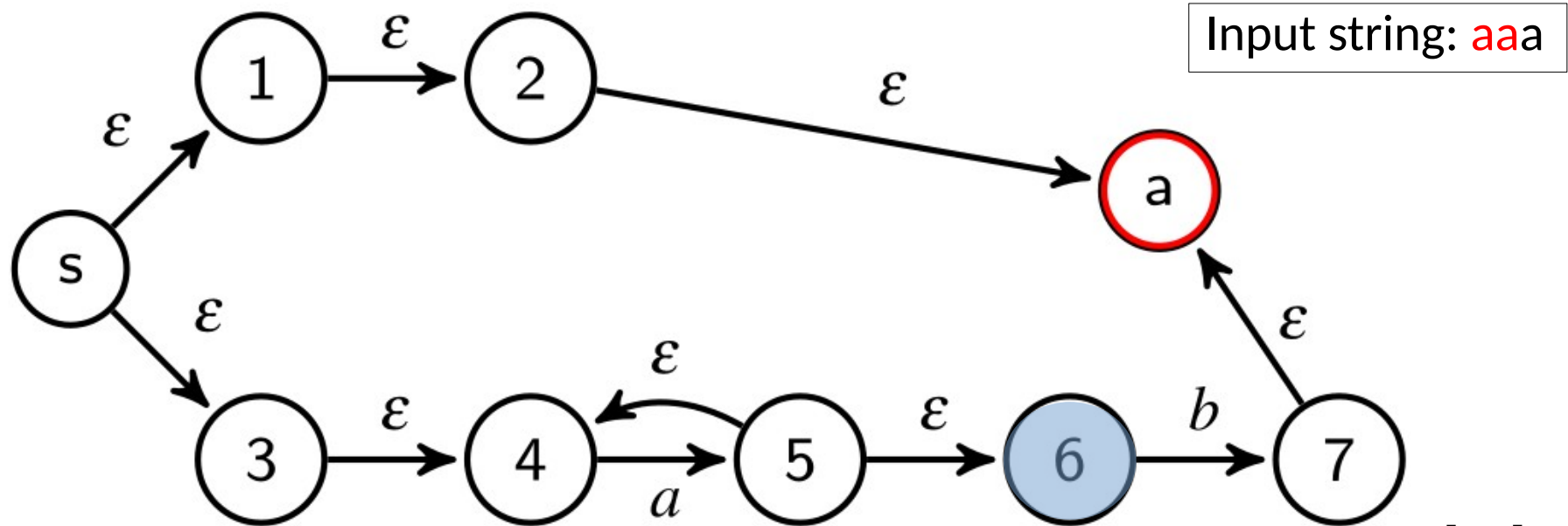
How does Spencer's algorithm work?

Convert $(aa^*b) \mid \epsilon$ into an NFA and then use backtracking



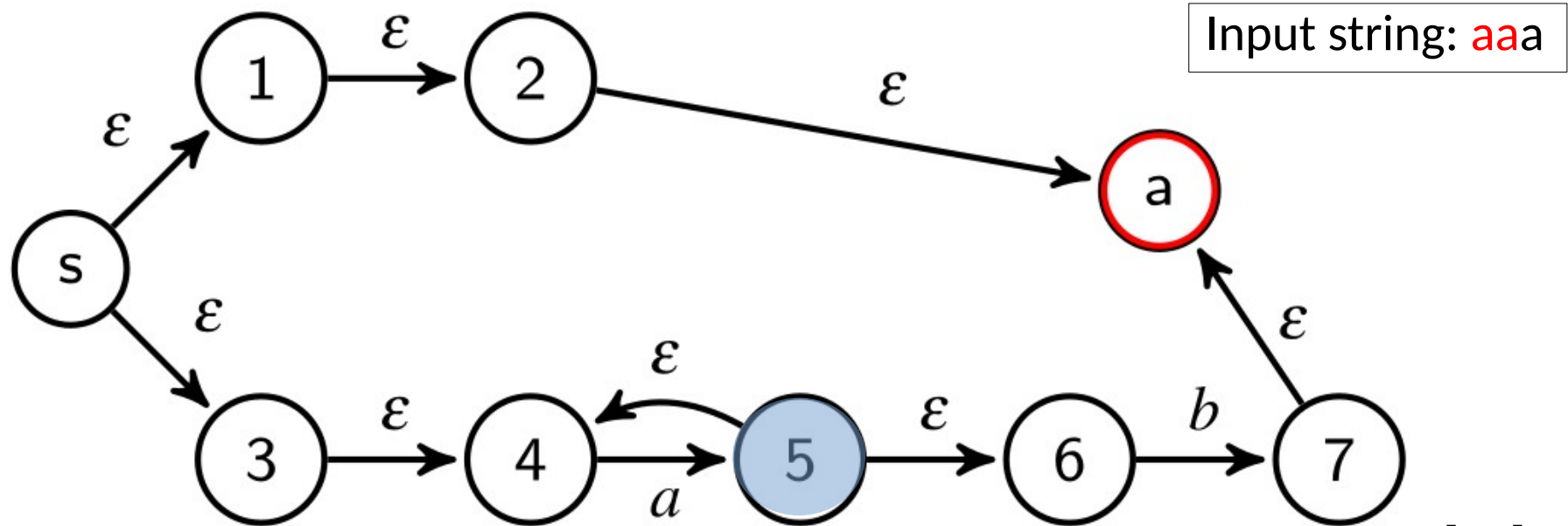
How does Spencer's algorithm work?

Convert $(aa^*b) \mid \epsilon$ into an NFA and then use backtracking



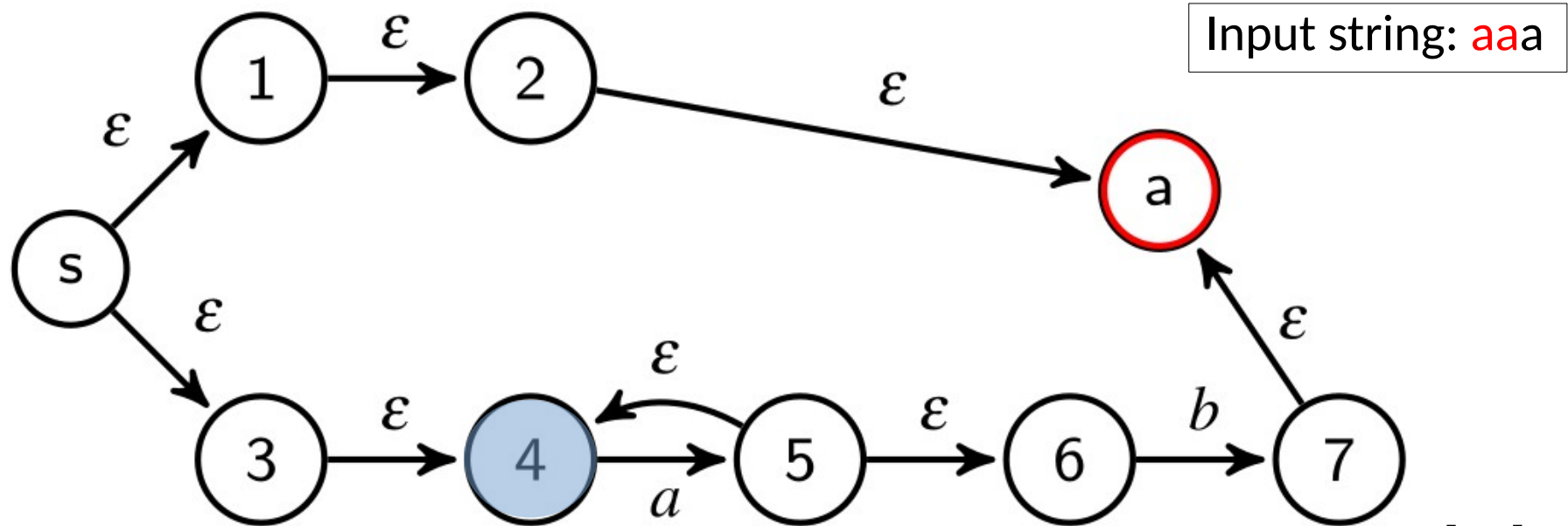
How does Spencer's algorithm work?

Convert $(aa^*b) | \epsilon$ into an NFA and then use backtracking



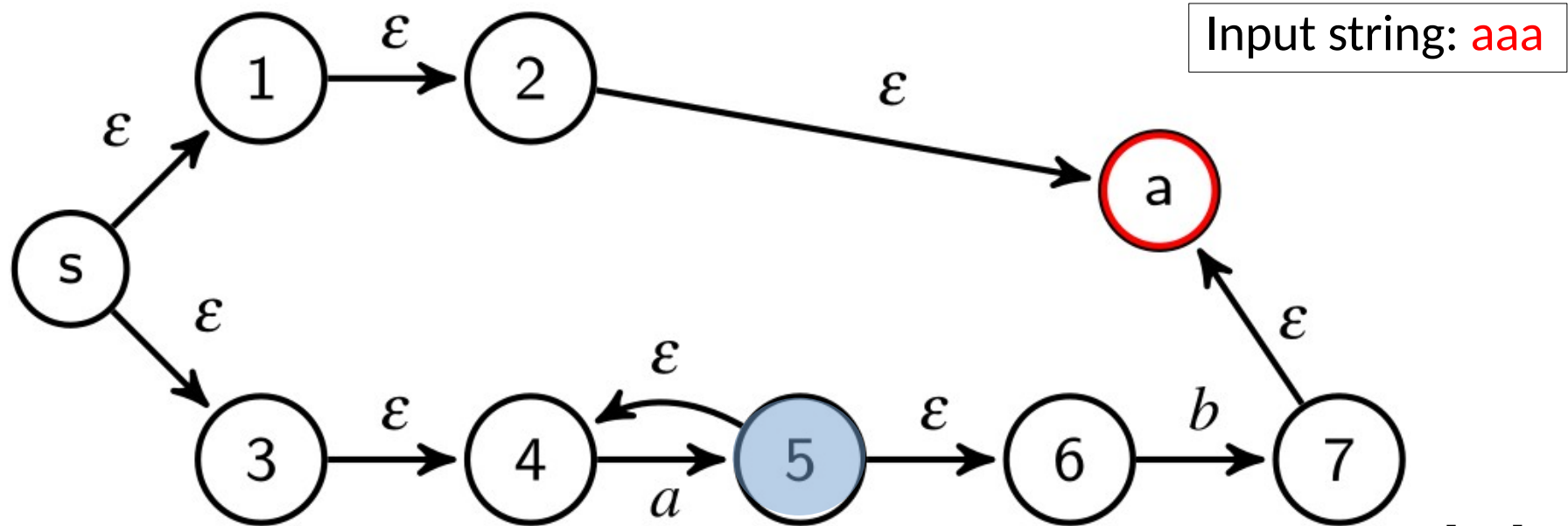
How does Spencer's algorithm work?

Convert $(aa^*b) \mid \epsilon$ into an NFA and then use backtracking



How does Spencer's algorithm work?

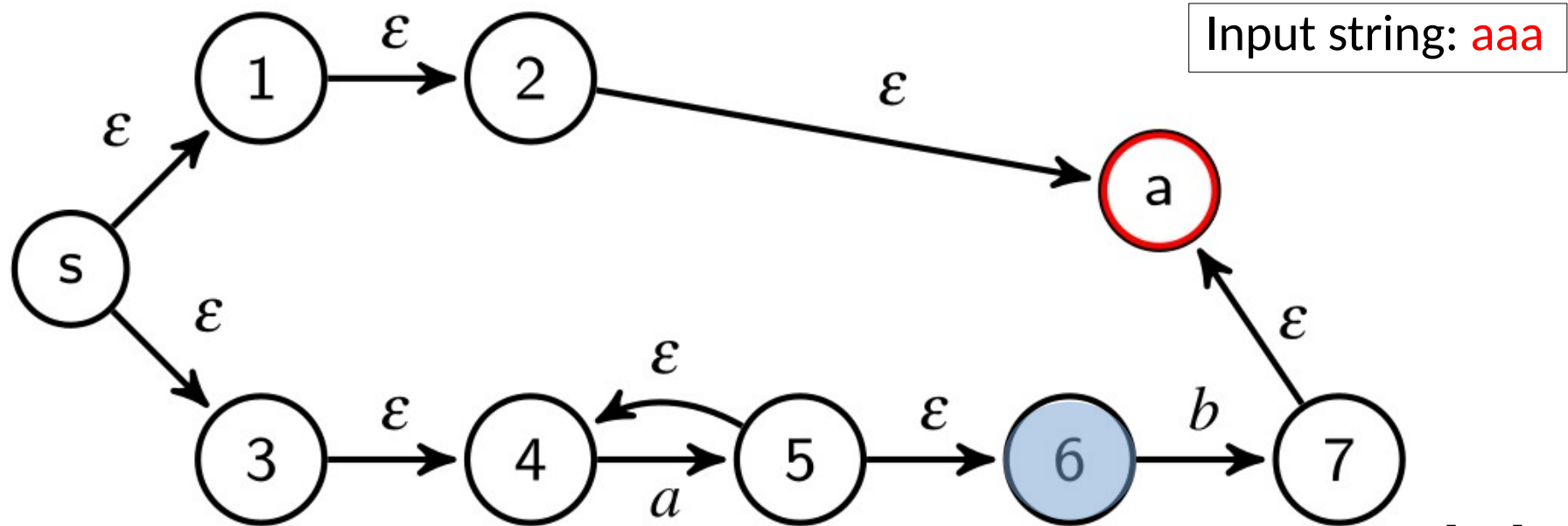
Convert $(aa^*b) \mid \epsilon$ into an NFA and then use backtracking



Input string: **aaa**

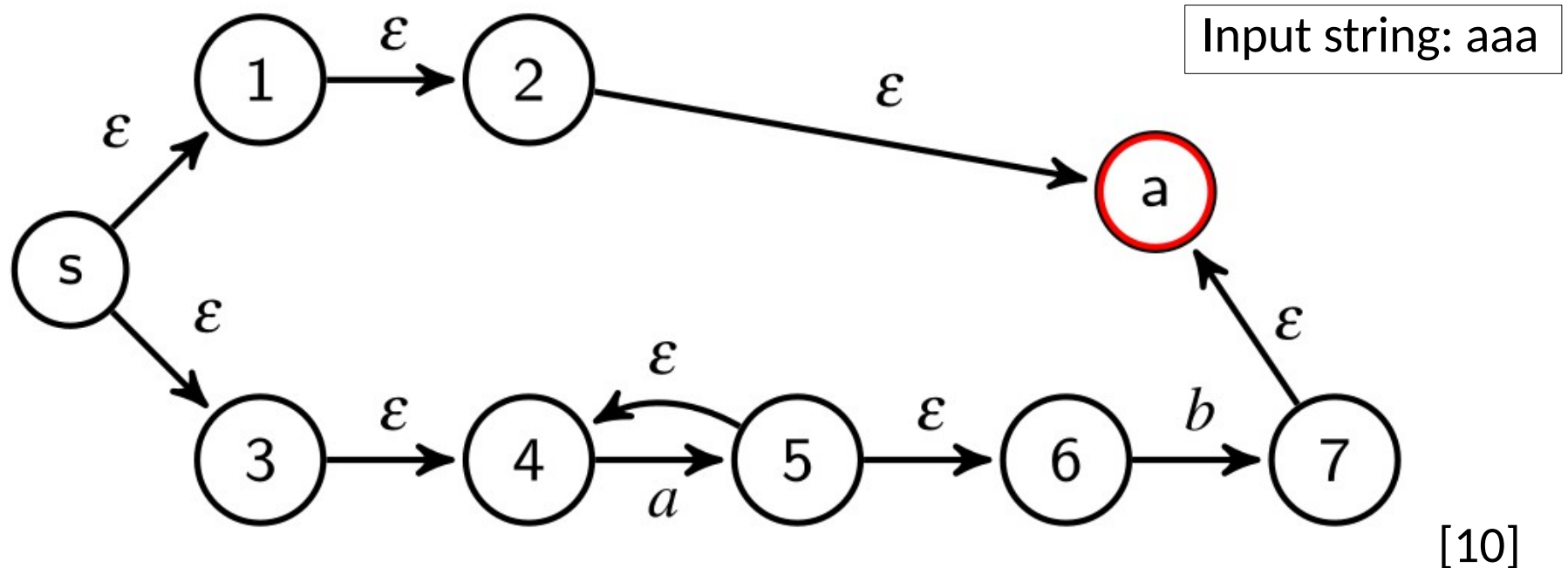
How does Spencer's algorithm work?

Convert $(aa^*b) \mid \epsilon$ into an NFA and then use backtracking



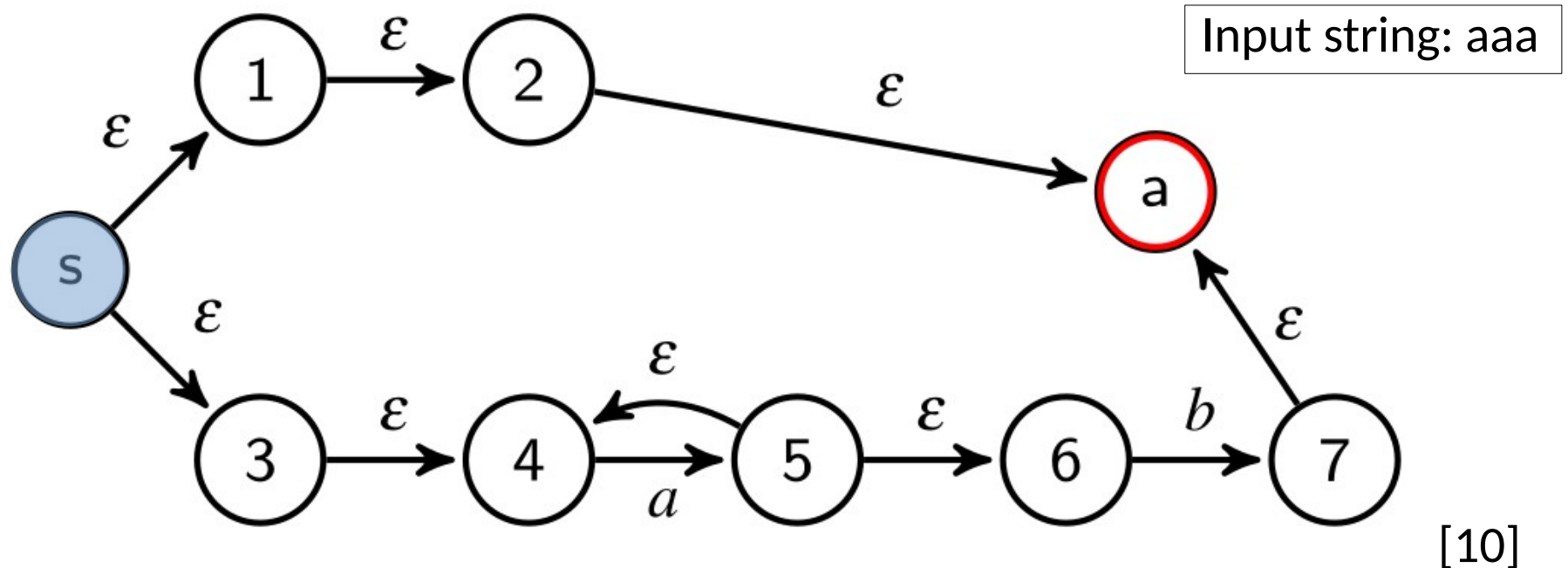
How does Thompson's algorithm work?

Convert $(aa^*b) | \epsilon$ into an NFA and then use concurrent search



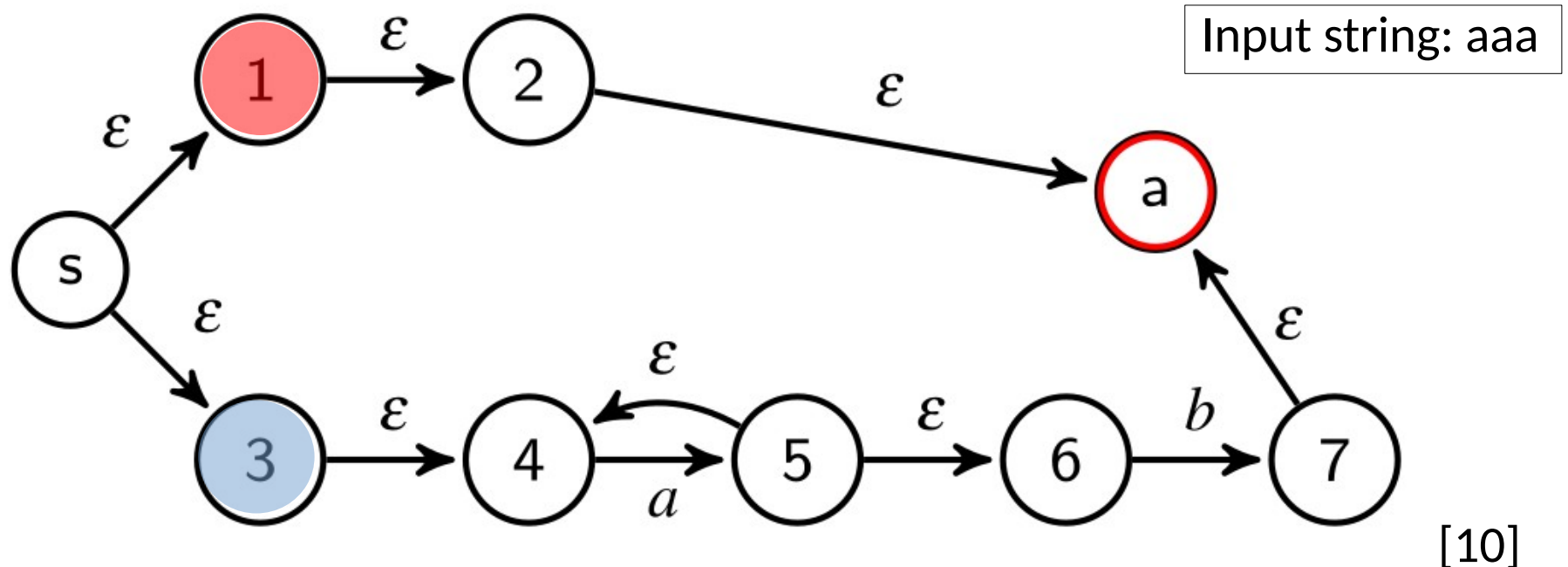
How does Thompson's algorithm work?

Convert $(aa^*b) \mid \epsilon$ into an NFA and then use concurrent search



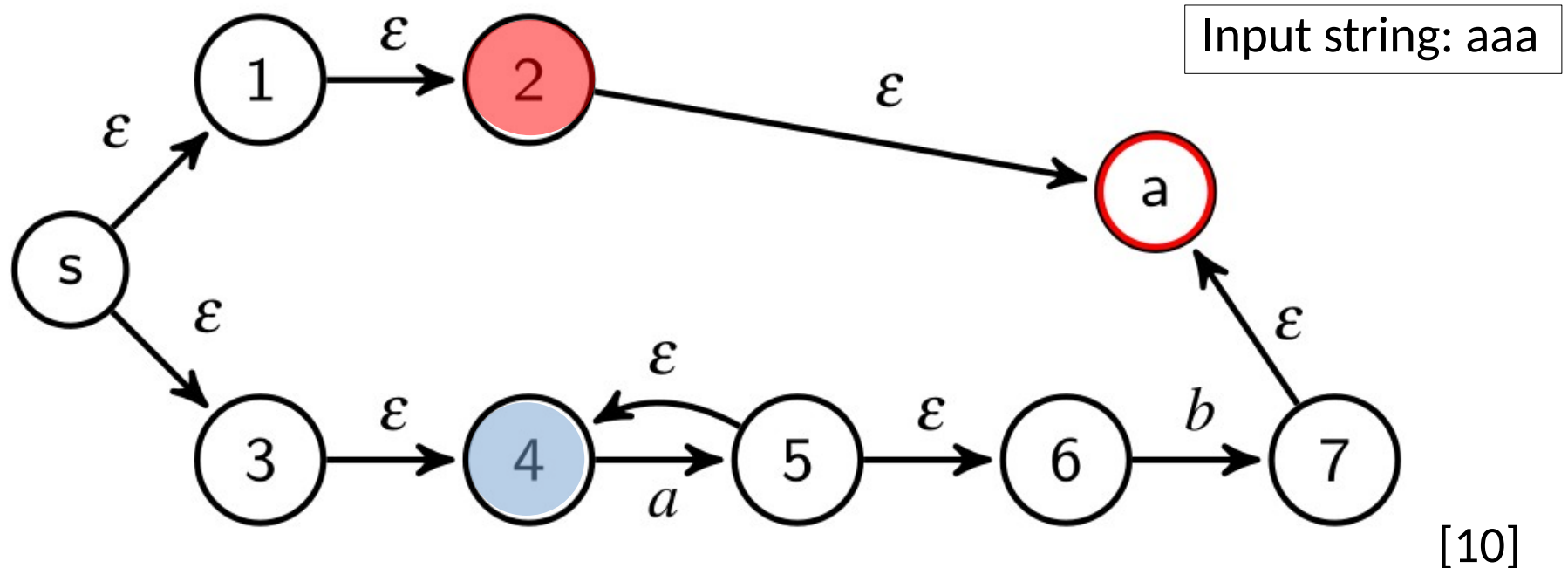
How does Thompson's algorithm work?

Convert $(aa^*b) \mid \epsilon$ into an NFA and then use concurrent search



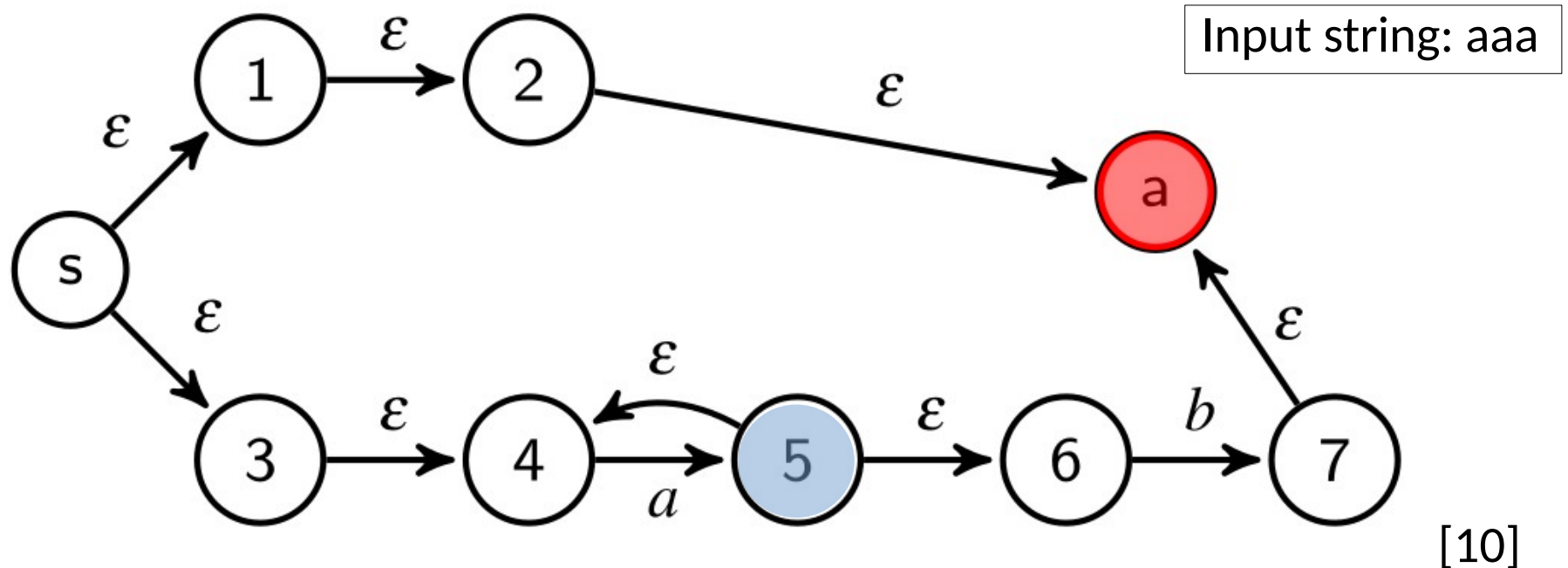
How does Thompson's algorithm work?

Convert $(aa^*b) | \epsilon$ into an NFA and then use concurrent search



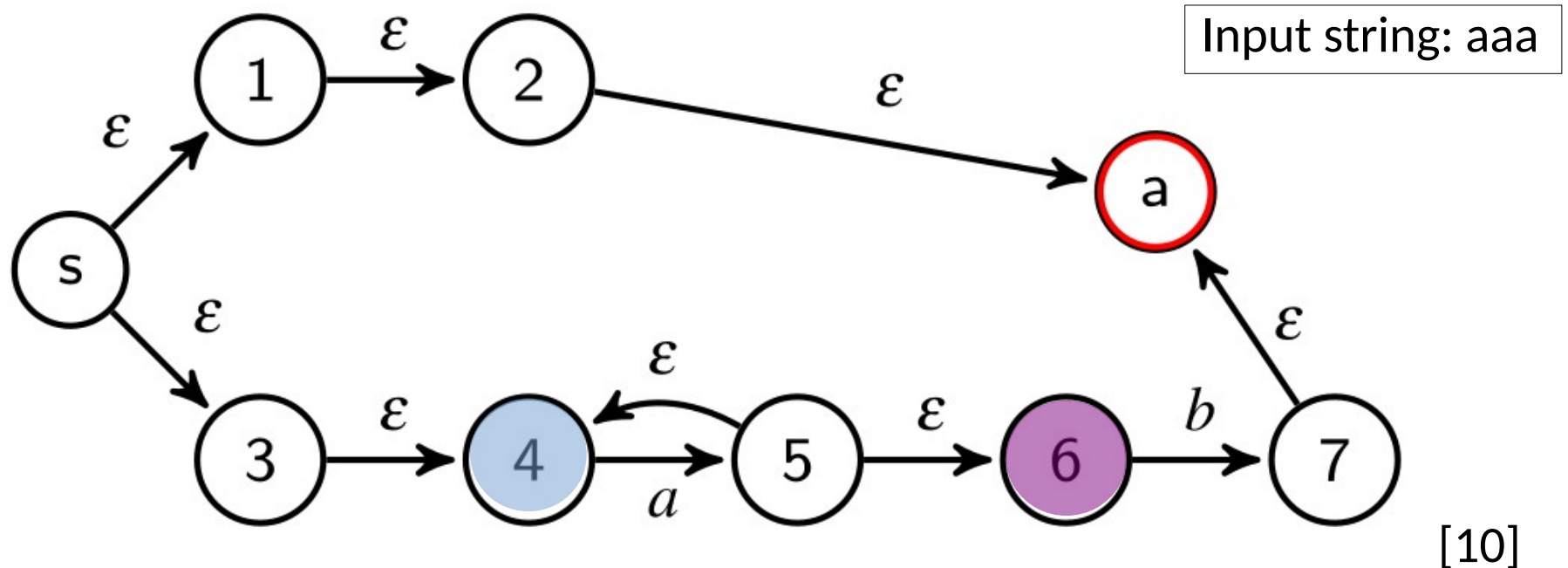
How does Thompson's algorithm work?

Convert $(aa^*b) \mid \epsilon$ into an NFA and then use concurrent search



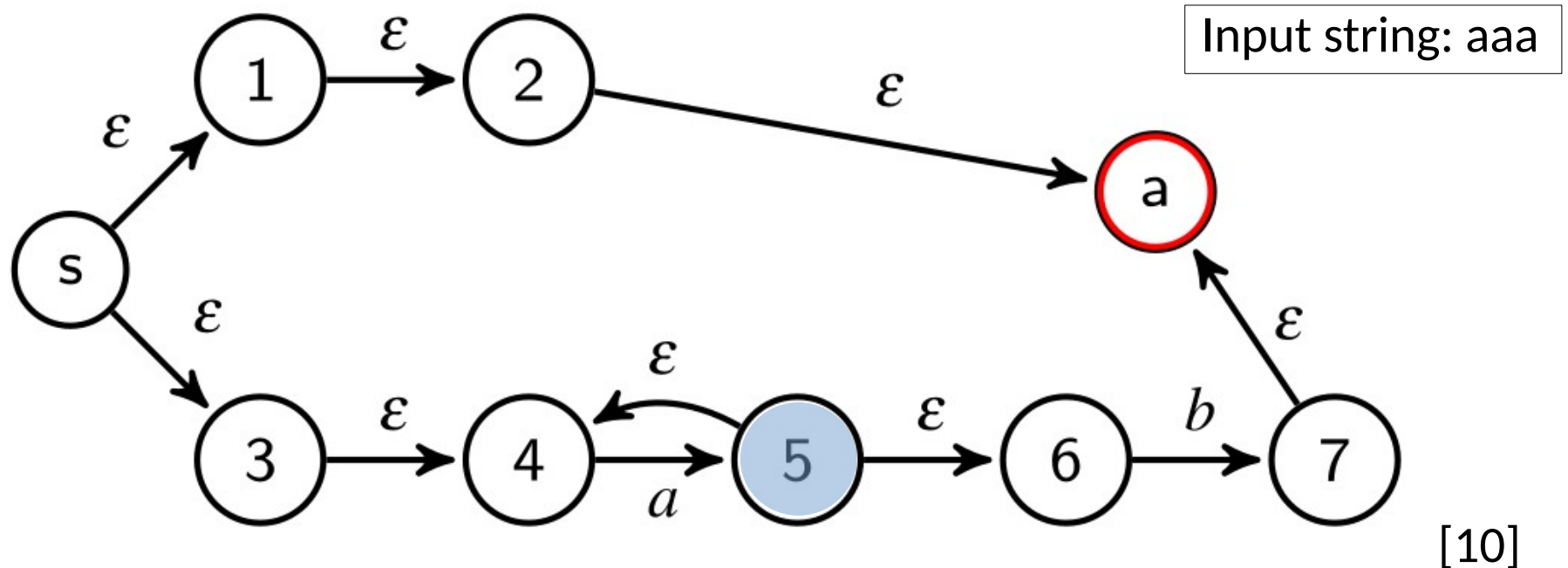
How does Thompson's algorithm work?

Convert $(aa^*b) | \epsilon$ into an NFA and then use concurrent search



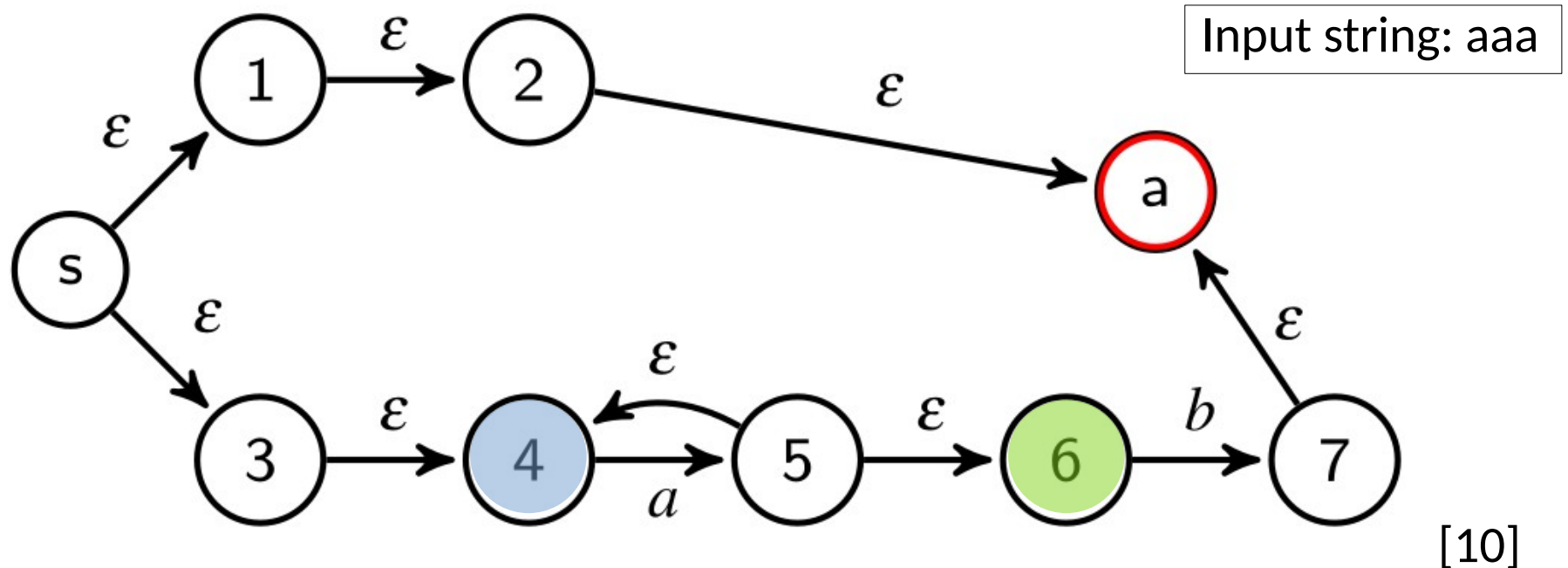
How does Thompson's algorithm work?

Convert $(aa^*b) \mid \epsilon$ into an NFA and then use concurrent search



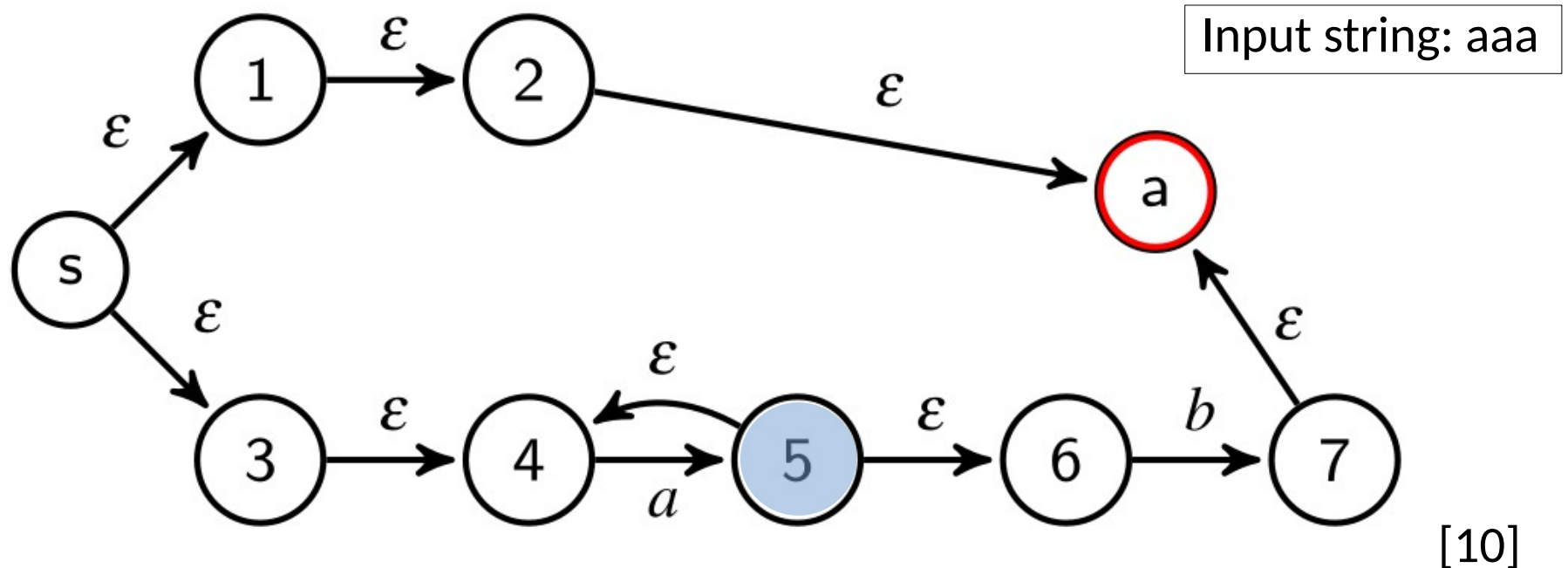
How does Thompson's algorithm work?

Convert $(aa^*b) | \epsilon$ into an NFA and then use concurrent search



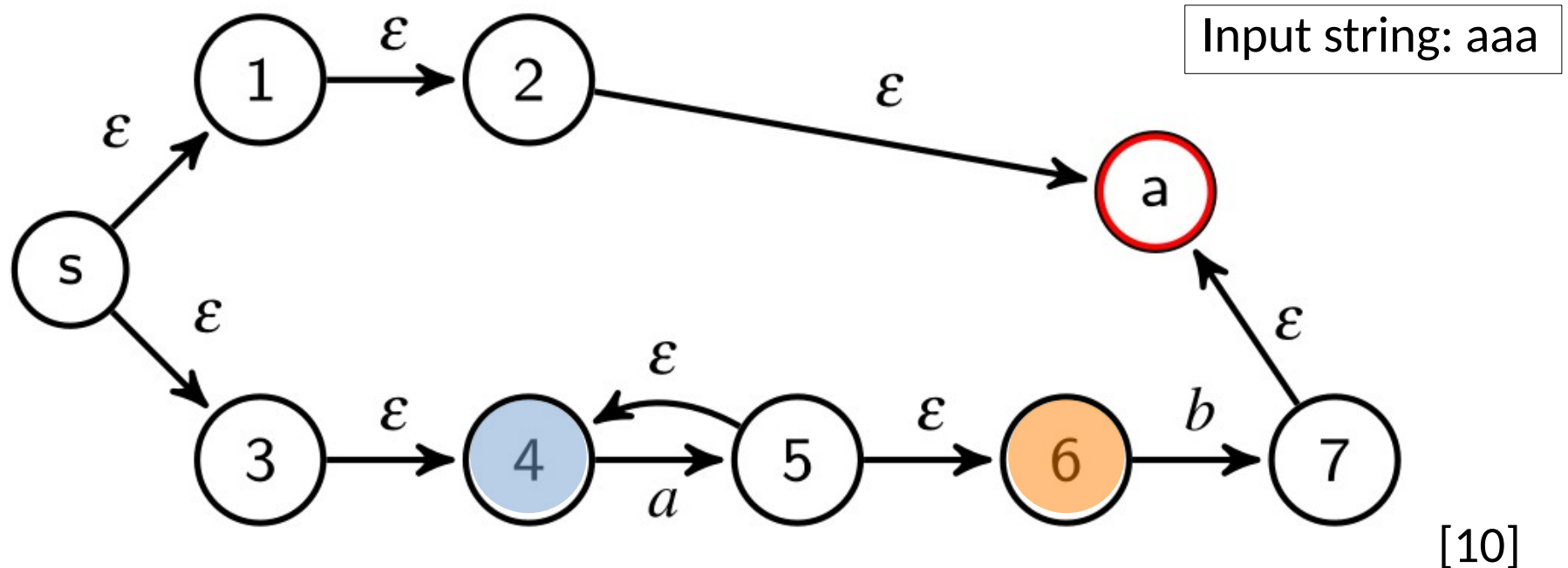
How does Thompson's algorithm work?

Convert $(aa^*b) \mid \epsilon$ into an NFA and then use concurrent search



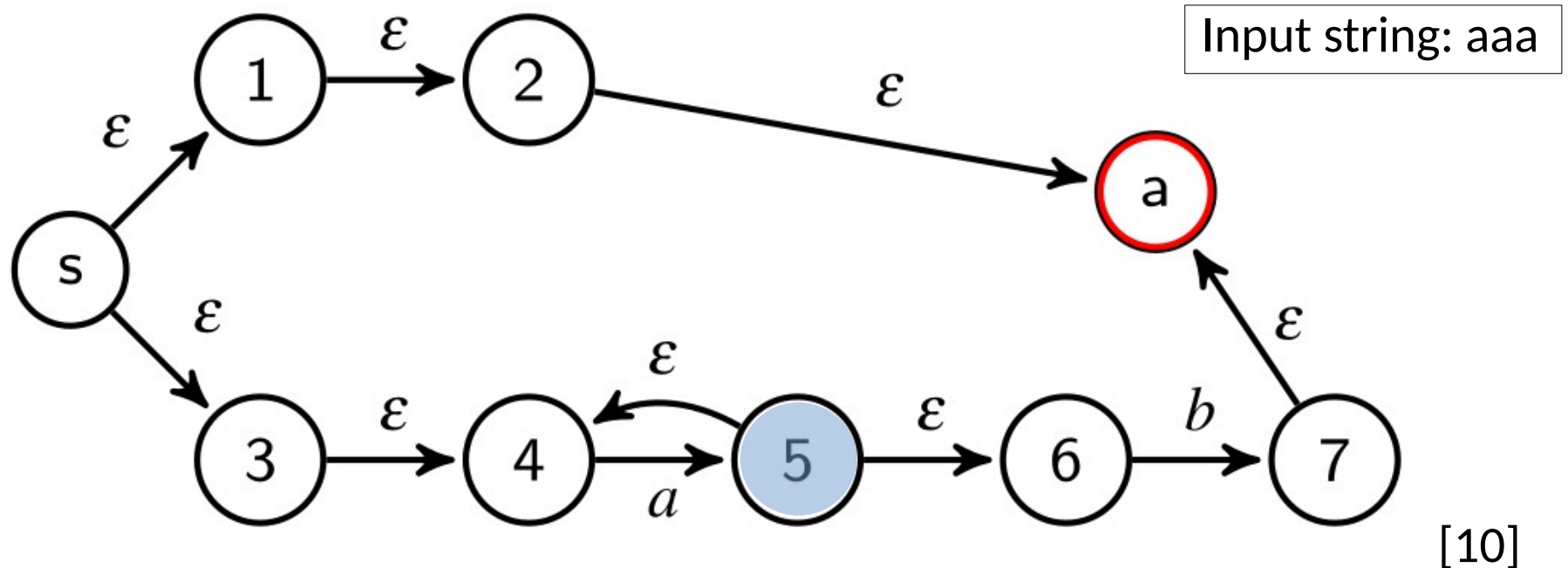
How does Thompson's algorithm work?

Convert $(aa^*b) \mid \epsilon$ into an NFA and then use concurrent search



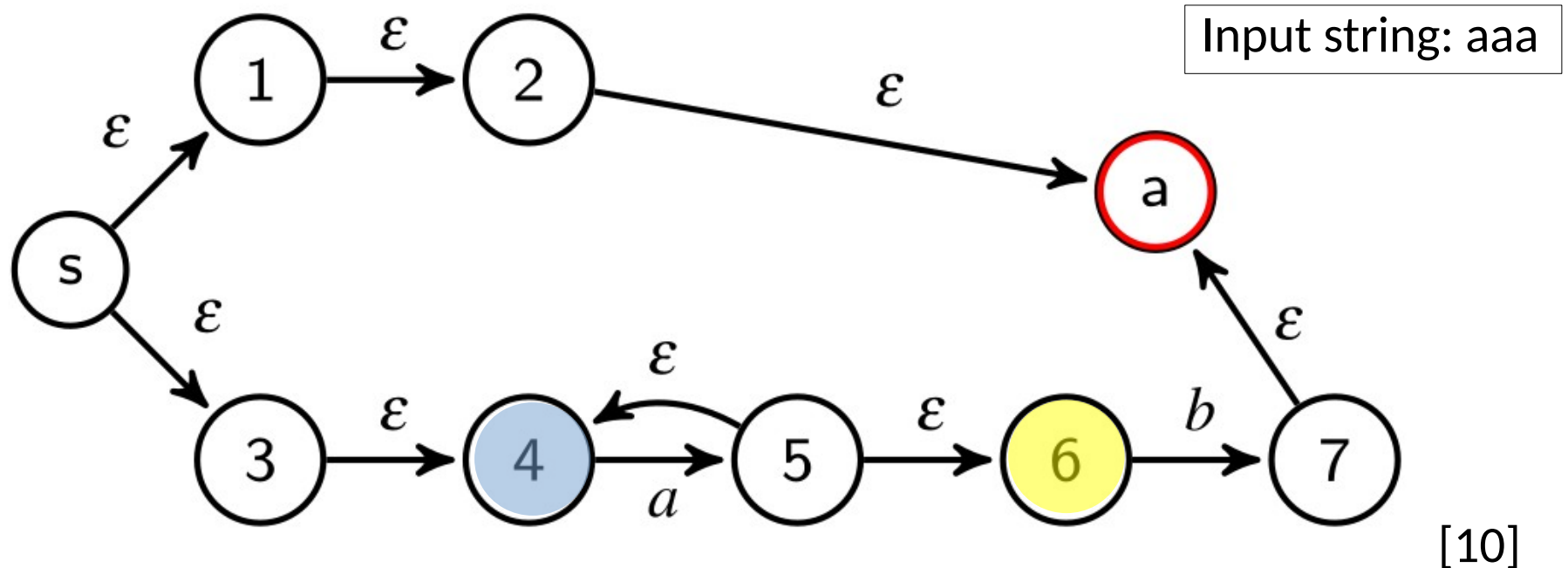
How does Thompson's algorithm work?

Convert $(aa^*b) \mid \epsilon$ into an NFA and then use concurrent search



How does Thompson's algorithm work?

Convert $(aa^*b) \mid \epsilon$ into an NFA and then use concurrent search



Agenda

1.

What is a regex?

2.

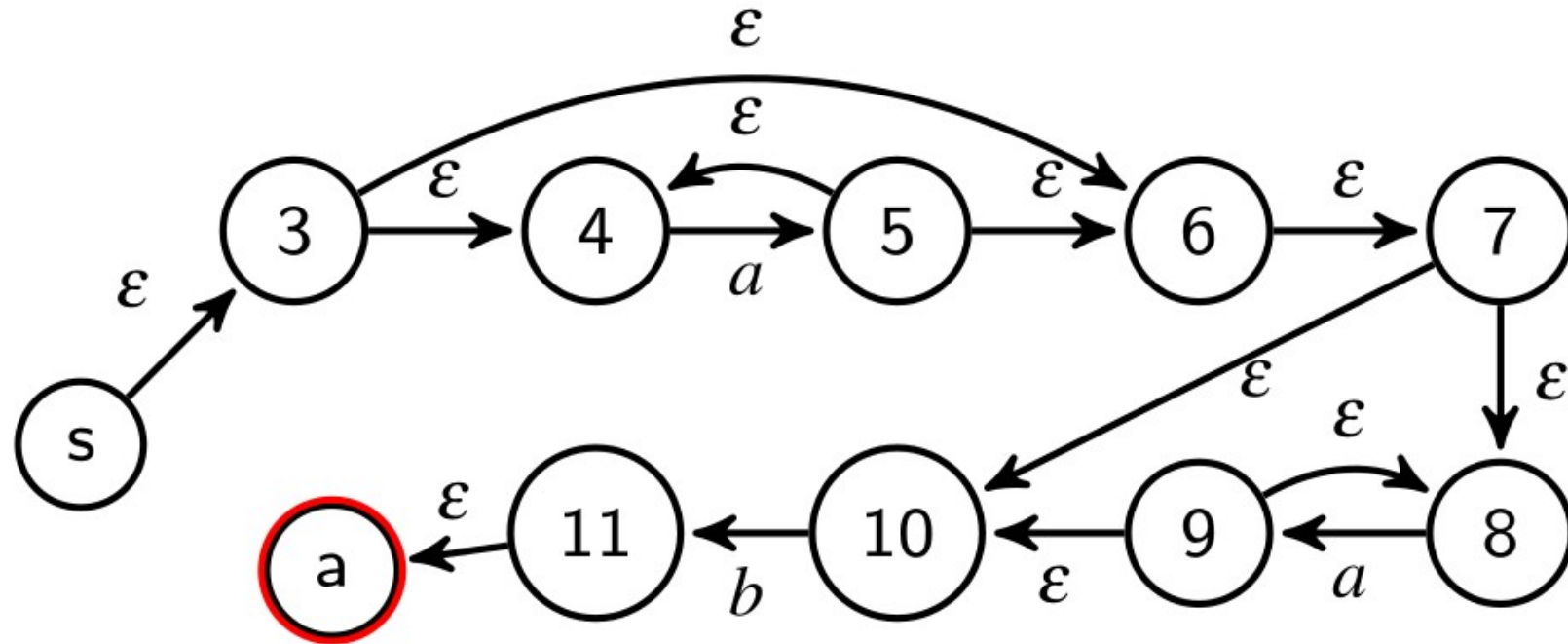
How are regexes
implemented?

3.

**How can regexes
lead to DoS?**

Why is Spencer's algorithm problematic?

Consider $\Sigma = \{a, b\}$, the regex a^*a^*b , and the input string
"aaaaaaaaaaaaa"



What are the ingredients of ReDoS?

Three components are necessary for ReDoS ^[4]:

1. The victim uses a backtracking regex engine
2. The victim uses a “vulnerable” (superlinear, potentially ambiguous) regex
3. The adversary is able to run user input against the regex
without resource limitations

What languages use backtracking?

A few years ago, ReDos was possible in ^[3]:

- JavaScript-V8 (Node.js)
- Python
- Java
- C++-11
- C#-Mono
- PHP
- Perl
- Ruby

Does ReDoS happen in real life?

Two major incidents:

1. Stack Overflow (2016)
2. Cloudflare (2019)

Neither of these incidents were due to malicious attackers.

Does ReDoS happen in real life?

Two major incidents:

1. Stack Overflow (2016)
2. **Cloudflare (2019)**

A new rule in Cloudflare's WAF contained a regex that caused every CPU core that handled HTTP/HTTPS traffic on the Cloudflare network worldwide to become exhausted ^[5].

```
(?: (?: \"|'|\]|\\}|\\\\|\\d| (?: nan|infinity|true|false|null|
undefined|symbol|math) |\\`|\\-|\\+)+[)]* ; ? ( (?: \\s|-|~|!|{|}\\|\\
\\||\\+)*\\.\\*(?:\\.\\*=\\.\\*) ) )
```


Does ReDoS happen in real life? ^[5]

- An engineer wrote a regular expression that could easily backtrack enormously.
- A protection that would have helped prevent excessive CPU use by a regular expression was removed by mistake during a refactoring of the WAF weeks prior.
- The regular expression engine being used didn't have complexity guarantees.
- The test suite didn't have a way of identifying excessive CPU consumption.

Is ReDoS a solved problem?

“The solution to this problem has been known since 1968 when Ken Thompson wrote a paper titled “Programming Techniques: Regular expression search algorithm”. The paper describes a mechanism for converting a regular expression into an NFA (non-deterministic finite automata) and then following the state transitions in the NFA using an algorithm that executes in time linear in the size of the string being matched against” [5].

Agenda



1.

What is a regex?

2.

How are regexes
implemented?

3.

How can regexes
lead to DoS?

What is a regex?

Over time, Kleene regexes have been extended to include syntax sugar, atomic groups, lookarounds, **backreferences**, etc. ^[4]

Agenda



1.

What is a regex?

2.

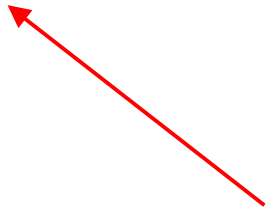
**How are regexes
implemented?**

3.

How can regexes
lead to DoS?

Why don't we use Thompson's algorithm?

1. Programming language designers are hesitant to make breaking changes to fundamental parts of the language ^[11].
2. Some extended features are much easier to implement using Spencer's algorithm ^[8].



Extended features are also a reason why DFAs are less common ^[9]

Agenda



1.

What is a regex?

2.

How are regexes
implemented?

3.

How can regexes
lead to DoS?

What is the current state of ReDoS? ^[4]

- JavaScript (Node.js-V8): Implemented Thompson-style regex engine for Kleene regexes (2021)
 - Remains opt-in for Node.js
- Ruby (MRI/CRuby): Implemented memoization technique for Spencer's algorithm as a defense (2022)
- Java (OpenJDK): Implemented a bounded caching mechanism (2016)
- C# (.NET): Implemented a timeout mechanism (2012) and a non-backtracking engine (2022) for Kleene regexes

What is the current state of ReDoS? [4]

- Python has neither timeouts nor built-in optimizations that protect against ReDoS
- Go and Rust have regex engines that were built with ReDoS in mind.

Exponential regexes behavior persists in Ruby and Java.
Polynomial behavior persists in JavaScript (and Java).

What is the current state of ReDoS? [4]

- Engineers disagree on whether ReDoS is a realistic risk.
- “[T]here has been no public description of a DoS attack that exploited ReDoS, so the risk posed by these vulnerabilities is still unclear.”
- Attackers (currently) tend to favor brute-force, volume-based distributed DoS attacks over weaponizing slow computations.

Summary



Questions?

References

- [1] A. Kamil, “Introduction to Computability” and “Finite Automata,” in *Foundations of Computer Science*, release 0.3, ch. 6-7, pp. 42-53. [Online]. Available: <https://eecs376.github.io/notes/>. Accessed: Dec. 18, 2025.
- [2] EECS 376 Instructional Staff, *Fall 23 Semester Lecture Slides*. University of Michigan, Ann Arbor.
- [3] J. C. Davis, C. A. Coghlan, F. Servant, and D. Lee. “The Impact of Regular Expression Denial of Service (REDOS) in Practice: an Empirical Study at the Ecosystem Scale”, in *Proceedings of the 26th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*, 2018.

References

- [4] M. H. M. Bhuiyan, B. Cakar, E H. Burmane, J. C. Davis, and C. Staicu, “SoK: A Literature and Engineering Review of Regular Expression Denial of Service”, in *Proceedings of the 20th ACM ASIA Conference on Computer and Communications Security (AsiaCCS)*, 2025.
- [5] J. Graham-Cumming. “Details of the Cloudflare outage on July 2, 2019.” Cloudflare. <https://blog.cloudflare.com/details-of-the-cloudflare-outage-on-july-2-2019> (accessed Dec. 17, 2025).
- [6] K. Thompson, “Programming Techniques: Regular expression search algorithm,” *Communications of the ACM (CACM)*, vol. 11, no. 6, pp. 419 – 422, June 1968, doi: 10.1145/363347.363387.

References

- [7] H. Spencer, “A regular-expression matcher,” in Software Solutions in C, USA: Academic Press Professional, Inc., pp. 35–71, 1994.
- [8] R. Cox, “Regular Expression Matching Can Be Simple And Fast (but is slow in Java, Perl, PHP, Python, Ruby, ...).” <https://swtch.com/~rsc/regexp/regexp1.html> (accessed Dec. 17, 2025).
- [9] “Understanding Regular Expression Engines: differences between non-deterministic (NFA) and deterministic finite automata (DFA).” The Gistre Blog. <https://blog.gistre.epita.fr/posts/alban.duval-mottet-2025-06-28-understanding-regular-expression-engines/> (accessed Dec. 17, 2025).

References

- [10] A. Staicu and M. Pradel, “Freezing the Web: A Study of ReDoS Vulnerabilities in JavaScript-based Web Servers”, in *Proceedings of the 27th USENIX Security Symposium*, 2018.
- [11] J. C. Davis, “Rethinking Regex Engines to Address ReDoS”, in *Proceedings of the 27th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE’19)*, 2019.